

A tertiary study on links between source code metrics and external quality attributes

Umar Iftikhar^{*}, Nauman Bin Ali, Jürgen Börstler, Muhammad Usman

Blekinge Institute of Technology, Department of Software Engineering, SE-371 79, Karlskrona, Sweden

ARTICLE INFO

Keywords:

Product quality
Quality models
Code quality
Evidence
Tertiary study
Tertiary review

ABSTRACT

Context: Several secondary studies have investigated the relationship between internal quality attributes, source code metrics and external quality attributes. Sometimes they have contradictory results.

Objective: We synthesize evidence of the link between internal quality attributes, source code metrics and external quality attributes along with the efficacy of the prediction models used.

Method: We conducted a tertiary review to identify, evaluate and synthesize secondary studies. We used several characteristics of secondary studies as indicators for the strength of evidence and considered them when synthesizing the results.

Results: From 711 secondary studies, we identified 15 secondary studies that have investigated the link between source code and external quality. Our results show : (1) primarily, the focus has been on object-oriented systems, (2) maintainability and reliability are most often linked to internal quality attributes and source code metrics, with only one secondary study reporting evidence for security, (3) only a small set of complexity, coupling, and size-related source code metrics report a consistent positive link with maintainability and reliability, and (4) group method of data handling (GMDH) based prediction models have performed better than other prediction models for maintainability prediction.

Conclusions: Based on our results, lines of code, coupling, complexity and the cohesion metrics from Chidamber & Kemerer (CK) metrics are good indicators of maintainability with consistent evidence from high and moderate-quality secondary studies. Similarly, four CK metrics related to coupling, complexity and cohesion are good indicators of reliability, while inheritance and certain cohesion metrics show no consistent evidence of links to maintainability and reliability. Further empirical studies are needed to explore the link between internal quality attributes, source code metrics and other external quality attributes, including functionality, portability, and usability. The results will help researchers and practitioners understand the body of knowledge on the subject and identify future research directions.

1. Introduction

Quality evaluation of source code artifacts such as test code and source code produced during development helps in identifying future risks and judging how well the software system will perform in use. Software quality evaluation is a context dependent and multidimensional concept [1]. Depending on the context and relevant perspective of quality, various measurement methods are utilized. Several definitions of software quality have been proposed in the literature. IEEE defines software quality as “the degree to which a system, component, or process meets specified requirements and customer or user needs or expectations” [2]. Software quality is often described as a set of properties possessed by the software itself. The presence or absence of these properties helps differentiate levels of software quality.

Similarly, several quality models have been proposed that define the properties of software using different terminologies into a set of factors, attributes, or characteristics, e.g., McCall et al. [3], Boehm et al. [4], Dromey [5] and ISO/IEC 25010:2011 [6] (see Section 2.1). While the models refer to similar basic properties, they may differ on how the properties are inter-linked, along with the terminology used to refer to these properties. Some researchers have used quality characteristics [7] while others preferred quality attributes [8] when describing properties of quality. Considering that more secondary studies use *quality attributes* to refer to quality factors, we use the same terminology for consistency.

Internal quality attributes of the product relate solely to the product without considering the environment of the product. On the other hand,

^{*} Corresponding author.

E-mail addresses: umar.iftikhar@bth.se (U. Iftikhar), nauman.ali@bth.se (N.B. Ali), jurgen.borstler@bth.se (J. Börstler), muhammad.usman@bth.se (M. Usman).

<https://doi.org/10.1016/j.infsof.2023.107348>

Received 27 June 2022; Received in revised form 13 September 2023; Accepted 8 October 2023

Available online 15 October 2023

0950-5849/© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

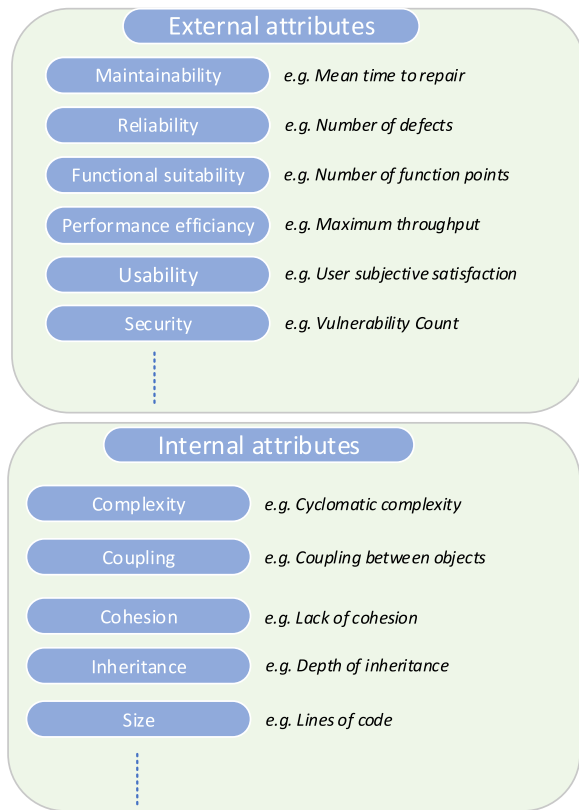


Fig. 1. Classification of internal and external code quality attributes, according to Fenton [9].

external quality attributes are described as properties of product behaviour in relation to the product's environment [9,10]. In the context of our tertiary study, the entity of interest is software source code. Internal quality attributes of source code can be assessed using code level metrics (e.g., lines of code) without executing the code. External quality attributes, such as how code behaves in a given environment or platform, can be measured using external metrics (e.g., number of defects), which are only available after execution. Assessing or predicting external quality attributes of software using internal quality attributes through source code metrics helps us measure product quality during development which can be invaluable considering the corrective costs related to post-deployment defects. The internal and external product attributes are classified in Fig. 1 along with examples of external metrics and source code metrics.

The subject of using source code metrics to assess external quality attributes has been studied in various empirical studies [11–13]. Similarly, a large number of secondary studies discuss utilizing source code metrics as indicators of external quality attributes [14–17]. While there are many secondary studies on the subject, the evidence on the effectiveness of source code metrics to assess or predict external quality attributes is varied and at times conflicting. We aim to synthesize the evidence reported in secondary studies and summarize the source code metrics that have been linked with external quality attributes.

The remainder of the paper has the following structure. Section 2 presents the related works in the field. Section 3 discusses the methodology used, followed by Section 4 on conducting the review. Results and analysis of the study are covered in Section 5 while Section 6 discusses the implications of the research. We discuss the limitations of the study in Section 7 and Section 8 concludes the study along with potential future work.

2. Related work

This section discusses four themes of related work for this study: relevant secondary studies on source code quality, software quality models, code quality models, and closely related tertiary studies.

2.1. Secondary studies on source code quality

Several secondary studies discuss source code quality in the context of software maintainability. The ISO/IEC 25010:2011 defines maintainability as one of eight top-level quality attributes.¹ Baldassarre et al. [16] present the results from 28 primary studies focusing on software models used in the context of software maintainability. Riaz et al. [15] reviewed 15 primary studies on software maintainability prediction using source code metrics. Abílio et al. [18] review 11 primary studies that focus on feature-oriented and aspect-oriented metrics that aid in measuring software maintainability. Saraiva et al. [17] provide a catalog of 570 maintainability metrics for object-oriented systems. Malhotra et al. [19] synthesizes 96 primary studies focusing on software maintainability prediction in the early stages of development. Burrows et al. [20] review 12 primary studies that use coupling metrics as an indicator of maintainability in the context of aspect-oriented programming. Nuñez et al. [21] provide a review of the current state of object-oriented metrics. Briand et al. [14] show that fault proneness of software may be predicted using size, coupling, and complexity metrics. We evaluate and synthesize the varying claims and evidence in the secondary studies linking source code metrics and external quality attributes.

2.2. Software quality models

The ISO/IEC 25010:2011 [6] defines the quality of software in terms of its attributes in a revised version of the previously established ISO/IEC 9126 [22] standard. International standards are important and common reference models to ascertain quality parameters that are relevant to a broad set of products for concerned stakeholders. The ISO/IEC 25010:2011 standard [6] defines software quality in terms of eight main quality attributes, further elaborated with 32 sub-attributes.

Several other quality models have been proposed. Grady et al. [23] have identified FURPS or functionality, usability, reliability, performance, and supportability as key attributes of quality. Soto et al. [24] define quality characteristics for open-source systems as maintainability, reliability, portability, operability, performance, functional suitability, security, and compatibility. Mayr et al. [25] propose a model to operationalize the ISO-9126 quality model with 336 code level metrics for embedded software. Another attempt at bridging the gap between ISO-9126 and the needs of the industry is presented by Manet [26] which introduces the concept of *practices* or guidelines to observe a group of metrics for better assessment of underlying code. An elaborate discussion on several quality models along with analysis on overlapping attributes is presented by Rashidi et al. [27] and highlight that quality models differ in terms of definitions of attributes and assessment methods.

2.3. Code quality models

An alternative model that supplements ISO/IEC 25010:2011 for code quality is provided by the Consortium of Information and Software Quality (CISQ) [28]. It measures source code quality in terms of four quality attributes: reliability, maintainability, security, and performance efficiency. Each of the quality attribute is evaluated in terms of a set of properties called common weaknesses and measured in terms of aggregated weaknesses or violations.

¹ ISO/IEC 25010:2011 refers to the same as quality characteristics.

SQALE [29] evaluates code in term of its testability, reliability, changeability, efficiency, security, maintainability, portability, reusability and is used by metrics tools such as SonarQube.²

2.4. Tertiary studies on related topics

The tertiary study by Lacerda et al. [30] discusses code quality from the perspective of code smells and code refactoring. The study analyses 40 systematic studies and shows that code smells affect six of the 32 sub-attributes of quality, as defined by the ISO/IEC 205010:2011. The affected attributes include understandability, maintainability, testability, complexity, functionality, and reusability. The paper lists duplicated code or duplicated clones as the most reported code smell. Source code metrics based methods are reported as one of the common methods to detect the top ten code smells. The report also summarizes popular code smell detection and refactoring tools that may aid practitioners. The study, however, does not investigate the link between source code metrics and external attributes and which source code metrics are good indicators of external quality based on the evidence in the secondary studies.

In this tertiary study, we aim to synthesize the evidence from secondary studies linking external quality attributes with source code metrics. The synthesis includes both qualitative and quantitative evidence reported in the secondary studies. Additionally, we also synthesize claims reported in secondary studies on the link between external quality attributes and source code metrics. Furthermore, we evaluate the evidence with strength of evidence criterion that takes into consideration several characteristics of the secondary studies. However, for this study, we exclude secondary studies that only report the use of source code metrics to assess external quality attributes. To the best of our knowledge, there are no tertiary studies that link source code metrics with external quality attributes reported in secondary studies.

3. Methodology

In this tertiary study, we pose and answer the following overarching research question using the guidelines by Kitchenham et al. [31]:

What is the current state-of-the-literature in terms of internal quality attributes and source code metrics that have been linked with external quality attributes and external metrics? A significant number of secondary studies exist on source code quality. Therefore, we answer this research question by identifying, evaluating and synthesizing such secondary studies to answer the following specific questions:

- RQ1:** *What are the characteristics of the secondary studies that link internal quality attributes and source code metrics with external quality attributes?*
- RQ2:** *What is the efficacy of the relationship between internal quality attributes and source code metrics with external quality attributes and external metrics?*
- RQ3:** *What is the efficacy of the models that use source code metrics to predict external quality attributes?*

In RQ1, we aim to cover the characteristics of the secondary studies, the years covered by the secondary study, the aims of the secondary studies, the type of systems, the size of systems, and the programming paradigm studied. In RQ2, we aim to provide a comprehensive evaluation of the qualitative and quantitative evidence-based links that associate internal quality attributes and their metrics with various external quality attributes and their metrics. In RQ3, we consider whether particular prediction models are better than others in predicting external quality attributes using source code metrics.

3.1. Search strategy

This section presents the strategy used for searching relevant secondary studies and formulating the search string. The search is designed to retrieve secondary studies that report evidence linking source code metrics with external quality attributes. In order to collect as many relevant secondary studies as possible, we preferred a broader search strategy to collect various forms of evidence. In this phase, we have not differentiated between secondary studies that only report usage of source code metrics and those that report evidence.

3.1.1. Keyword-based search

We used keyword-based [31,32] search in one indexing (i.e., Scopus) and two publisher databases (IEEE Xplore and ACM digital library) as recommended by Petersen et al. [33]. IEEE and ACM publish the most important journals and conferences in the software engineering field [34,35]. Moreover, Scopus is considered as one of the largest indexing services covering papers from ACM, IEEE, Springer, Wiley and Elsevier [36,37].

3.1.2. String construction

We used keywords and synonyms from ISO/IEC 25010:2011 [6], related key-words from a known set of papers (see online.³) and our domain knowledge. The search string used in the study has the following three clusters of keywords:

Artifact: code, software program, software product, software application, software system, object-oriented, aspect-oriented, feature-oriented.

Quality: quality, smell*, pattern, functional suitability, performance, efficiency, compatibility, usability, reliability, security, maintainability, portability, analyzability, modifiability, testability, compliance, stability, comprehension, understandability, understanding, maintenance, modularity, reusability, changeability, evolvability, modification, evolution, readability, metric*, measur*, indicator, refactoring

Systematic secondary study: systematic review, systematic literature review, systematic map, systematic mapping, tertiary study, tertiary review, mapping study, multivocal literature review, multivocal literature mapping.

The search string was formed by combining the above sub-strings with a Boolean AND operator as depicted online.³ We did not consider secondary studies shorter than eight pages as they are unlikely to contain necessary details related to evidence between source code metrics and external quality attributes. We restricted ourselves to papers published in peer-reviewed conferences, and journals to report the evidence widely accepted by the research community. We executed the search string in February 2021.

Moreover, in the final set of publications considered for selection, we added the publications from the validation and known sets of publications (see Section 3.1.3), and the secondary studies identified by Lacerda et al. [30].

3.1.3. Search validation

To evaluate the effectiveness of the search string, we used a set of secondary studies (see online³) as a quasi-gold standard (QGS) [31]. Two authors, not involved in the design of the search string, independently formulated the validation set which comprised 11 secondary studies [38].

Our search strategy achieved a recall of 73% which is moderate. The recall could further be improved with additional keywords. However, this led to an increase in the number of results that was impractical. Moreover, as we have supplemented the keyword-based search with relevant secondary studies from the tertiary studies by Lacerda et al. [30], we decided not to make this change.

² <https://www.sonarqube.org/>

³ <https://doi.org/10.5281/zenodo.7933498>

Table 1
Inclusion/exclusion criteria used in the tertiary study.

Inclusion criteria	
C0	Publications in English language and with length of at least eight pages
C1	Peer-reviewed workshop, journal or conference publications
C2	Publications claiming to have systematically studied available literature, i.e., systematic literature studies (SLRs or SMSs) or multivocal literature studies (MLRs, MLMs)
C3	Papers that identify, define or measure internal quality attributes or determine levels of code quality (e.g., work on quality measurement or code smells)
C4	Papers that relate code metrics/quality attributes/code refactoring/code smells to external quality attributes
Exclusion criteria	
C5	Publications that are about only external quality attributes of software product/system/service, or about the quality of other artifacts like defect reports, test code, or test cases

Table 2
Decision rules in selection strategy based on Ali and Petersen [39].

		Reviewer 2		
		Relevant	Maybe	Irrelevant
Reviewer 1	Relevant	Include	Include	Resolve with discussion
	Maybe	Include	Include	Exclude
	Irrelevant	Resolve with discussion	Exclude	Exclude

3.2. Selection process

This section discusses the steps of the selection process.

3.2.1. Selection criteria

To identify relevant papers from the search results, we used the criteria listed in Table 1. Papers fulfilling the boolean expression (C0 AND C1 AND C2 AND (C3 OR C4)) were selected for full-text reading. In unclear cases, we were inclusive, i.e., when there were indications that the full-text of a paper might contain information fulfilling C3 or C4, we retained the paper for the next phase. Papers that did not fulfil C3 or C4 but fulfilled C5 were excluded.

3.2.2. Preliminary selection

Before applying topic specific selection criteria, the first author excluded publications with less than eight pages and not written in English.

3.2.3. Selection based on title and abstract

To check the objectivity of the criteria and whether we have a shared understanding of it, we performed a pilot round of the selection process [31,39]. The piloting step involved all four authors. 12 randomly selected papers from the search results were assessed independently by all authors as *relevant*, *irrelevant* or *maybe relevant*. Cohen's Kappa measure of inter-rater agreement was used to measure the level of agreement between authors. We aimed for an inter-rater agreement above 0.61 since it is considered a substantial agreement in literature [40,41].

Any disagreements during piloting provide an opportunity of reviewing or updating the selection criteria in an iterative manner [33]. We resolved all disagreements through discussion and referring back to the selection criteria.

3.2.4. Selection based on adaptive reading

As stated in Section 3.2.1, we have been inclusive when the titles and abstracts have been insufficient to conclude the relevance of a paper. To deal with such papers in an efficient manner, we have used a modified adaptive reading method [42]. We first read the research

questions of the secondary study to know more about its aims and objectives, followed by the introduction and conclusion sections to decide its relevance to our study. The selection criteria listed in Section 3.2.1 were used to ascertain the relevance. Any papers excluded during this process were reviewed by the second author to reduce the likelihood of excluding a relevant paper.

3.2.5. Selection based on full-text reading

In this step, the full-text of the papers was read to extract the information required to answer the research questions. During this step, an explicit attempt was made to identify overlapping papers [43], i.e., multiple papers reporting the same or extended version of a study.

3.3. Data extraction

Table 3 presents the data extraction form and a mapping of the data to respective research questions.

3.3.1. Piloting of data extraction

A piloting of the data extraction form was done to assess its reliability and completeness. The first and third author independently extracted the data from a randomly selected secondary study from the validation set to reduce the chance of selecting a secondary study that lacks sufficient data to validate the data extraction form. Any differences were resolved through a discussion.

3.3.2. Data extraction validation

After the complete data extraction, six secondary studies were randomly selected for validation of the extracted data by the third author. Due to the laborious nature of extracting multiple entries of measures and metrics from secondary studies, it was decided to mark the relevant section/tables for internal/external quality attributes, and reported source code metrics.

3.4. Quality assessment of the secondary studies

The five DARE⁴ questions for the quality assessment of systematic reviews in healthcare have been widely used in software engineering research [44]. For this tertiary study, the criteria proposed by Budgen et al. [45] to answer the five DARE questions were used (see Table 19 in the appendix).

3.4.1. Piloting of the quality assessment criteria

The quality assessment form was piloted on two randomly selected secondary studies from our sample by the first and fourth authors.

3.4.2. Quality assessment validation

The first author performed the quality assessment for all included secondary studies. Six secondary studies were randomly selected for validation by the fourth author.

As recommended by Kitchenham et al. [31], the quality assessment score can be utilized to remove low-quality secondary studies. This approach has been used by other tertiary studies such as Hoda et al. [46], Barros-Justo et al. [36] and Curcio et al. [47]. Inspired by [46,47], we removed secondary studies that score ≤ 1.5 (of 5), i.e., removed the papers that answered “no” to at least two questions and scored “partial” (0.5) on other questions.

The quality DARE score of a secondary study is also used during synthesis of results as an indicator of strength of evidence (see Section 3.5.4 and Table 5). The DARE score of a secondary study

⁴ The Centre for Reviews and Dissemination (CRD) suggests five questions to determine whether to include a systematic review in their Database of Abstracts of Reviews of Effects (DARE). <https://www.crd.york.ac.uk/CRDWeb/AboutPage.asp> accessed on: April 17, 2022.

Table 3

Data extraction traceability with research questions.

Research question	Data extracted
RQ1: What are the characteristics of the secondary studies that link internal quality attributes and source code metrics with external quality attributes?	– Metadata: (author, title, publication venues, publication date) – Aims: (research questions of the secondary study) – Search: (time period covered in the search, databases used). – Code quality attribute that are the secondary study's focus: (from the research questions) – Type of software system reported by the secondary study – Programming paradigm reported by the secondary study – Application domain considered by the secondary study – Software size reported by the secondary study – Type of research method used in secondary studies – Quality of the secondary link: selection criteria, search coverage, quality assessment, study description and synthesis
RQ2: What is the efficacy of the relationship between internal quality attributes and source code metrics with external quality attributes and external metrics?	– Name of external quality attribute/sub-attribute – Name and description of the external metric – Linked to which internal attribute – Name of source code metrics used to measure the external attribute – Description or definition of the source code metrics – Validation status of the source code metrics – Number of primary studies reporting the link between source code metrics and external quality attributes – Evidence for the reported link (e.g., claims, correlation results, prediction results)
RQ3: What is the efficacy of the models that use source code metrics to predict external quality attributes?	– Models used for prediction of external quality attributes – Accuracy of the prediction models – Datasets used for validation – Source code metrics used for prediction

is classified as high, medium or low using the suggestion by Curcio et al. [47]. To critically appraise the quality assessment performed by the included secondary studies, we categorized the quality assessment questions used in those secondary studies. The categorized quality assessment questions primarily assess the quality of reporting. However, focusing on methodological issues during critical quality appraisal is more beneficial [44,48]. More recent proposals, like QAISER [49] for appraising systematic reviews, operationalize such assessments by going beyond reporting quality and assessing the risk of bias. For each included secondary study, we calculate an additional quality assessment score besides the DARE score. This additional quality assessment score is computed as follows: we count the number of categorized quality assessment questions used by the secondary study.

3.5. Collation process: attributes & metrics

This section reports the methodology used when collating extracted data related to internal attributes, external metrics, and source code metrics, along with the strength of evidence.

3.5.1. Attributes and metrics

The external quality attributes linked to the source code metrics are based on the information available in the secondary study regarding the external metric or dependent variable. Similarly, if reported by the study, the internal quality attribute is extracted. In the case where a secondary study has not assigned any internal quality attribute, and the source code metric belongs to a well-known metric suite, we then assign the internal quality attribute based on the description of the metric. If the metric is not part of a well-known suite, we then attempt to extract the description from the primary study quoted by the secondary study.

3.5.2. Relationship indicators quality attributes and source code metrics

The secondary studies report different types and levels of evidence when linking source code metrics to external quality attributes. We mapped the different levels of evidence reported in secondary studies as “++”, “+”, “0”, “-”, “--” and “unclear”. For the variety (e.g. correlation results, prediction accuracy results) of evidence reported, we interpret “++” as ‘strong positive relationship’, “+” as ‘positive relationship’, “0” as ‘insignificant relationship’, “-” as ‘negative relationship’, “--” as ‘strong negative relationship’ and “unclear” as ‘unclear relationship’ (see Table 4). Different types of evidence include explanatory secondary studies (i.e., studies predicting external quality attributes using source code metrics) report accuracy measures,

while exploratory secondary studies (i.e., studies investigating the relationship between external quality attributes and source code metrics) report correlation co-efficients or claims of relationship.

3.5.3. Double counting

Double counting can occur during results aggregation in tertiary studies when the result from the same primary study is reported by more than one secondary study and is counted more than once. If more than one secondary study reports the evidence of a link between source code metric and external quality attribute used, we also investigate if there is a risk of double counting. Thus, we consider whether these secondary studies share any primary studies when reporting their results. While processing the degree of overlap, we only considered overlap in primary studies that are used to report results in the secondary studies (thus if a primary study is found in more than one secondary study, but only one secondary study utilizes it as a source for its results while the others consider it as reference is not considered an overlap).

In case there is no overlap, we report “No overlap”. We note the primary study title, the author, and the year if overlap exists. We match these details within the other secondary study to see whether the other secondary study also uses results from the same primary study. If this is true, then based on how the secondary study results are presented, we try to remove the overlap before reporting the results (by counting results only once in the overlapped primary studies). If this is possible, we adjust the result to count it only once and report “X PS, Resolved” where X represents the number of removed studies. In case this is not possible, we report the overlap present and use it as a factor to downgrade the strength of evidence (see Section 3.5.4).

3.5.4. Strength of evidence

To indicate the strength of evidence for the relationship between external quality attributes and source code metrics discussed in the previous sections, we considered twelve factors: (1) the DARE quality score of the secondary study, (2) the aggregate number of primary studies (within the secondary studies) reporting the stated link between source code metric and external quality attribute, (3) whether the secondary studies have reported and described the external metric to measure the external quality attribute, (4) the degree of overlap among secondary studies reporting the same result, (5) the additional quality assessment score calculated in Section 3.4.2, (6) whether the secondary studies reported the source code metrics used as validated, (7) whether the secondary studies have provided a replication package or reported the extracted data in the annexure, (8) whether the authors

Table 4

Labels used indicate a relationship between external quality attributes and source code metrics.

Assigned label	Evidence reported by secondary study
Exploratory secondary studies	
++	Strong positive correlation (with or without significance levels reported) Strong claims of positive relation
“+”	Positive correlation (significance levels reported) Positive correlation (no significance levels reported) Claims of positive relation
“0”	Insignificant results (significance levels reported) Insignificant correlation (no significance levels reported) Claims of insignificant relation
“-”	Negative correlation (significance levels reported) Negative correlation (no significance levels reported) Claims of negative relation
“--”	Strong negative correlation (significance levels reported) Strong negative correlation (no significance levels reported) Strong claims of negative relation
“Unclear”	Unclear results (significance levels reported) Unclear results (no significance levels reported) Claims of unclear relation
Explanatory studies	
“+”	Accuracy measures (e.g., TPR, FPR, Mean ARE, AUC) reported by secondary study as successful predictors Accuracy measures (e.g., MMRE) interpreted by our tertiary study as good predictors using [50,51]
“0”	Accuracy measures reported by secondary study as insignificant predictors Accuracy measures (e.g., MMRE) interpreted by our tertiary study as insignificant predictors using [50,51]
“-”	Accuracy measures reported by secondary study as unsuccessful predictors Accuracy measures (e.g., MMRE) interpreted by our tertiary study as unsuccessful predictors using [50,51]
“Unclear”	Accuracy measures reported by secondary study as unclear predictors Accuracy measures (e.g., MMRE) interpreted by our tertiary study as unclear predictors using [50,51]

have explicitly reported any conflict of interest, (9) whether secondary studies reporting correlations results have reported if their primary studies have adjusted the results for using multiple statistical tests. We consider three additional criteria for prediction-focused secondary studies in addition to the nine criteria, (10) whether unbiased or unskewed evaluation measures were used to report the quantitative evidence in secondary studies, (11) whether the secondary studies that focus on fault-proneness or defect prediction reported normalization by the size of modules in their primary studies, and (12) for secondary studies that focus on prediction, if the data used in primary studies (as reported by secondary studies) is post-hoc or do the secondary study also report results of prediction of future faults/defects/change as well. The criteria used are summarized in Table 5.

4. Conducting the review

The results of the overall selection process are summarized in Fig. 2. Individual steps of the process are described in the following sub-sections.

4.1. Search results

Fig. 2 summarizes the number of results from individual databases and the total number of unique secondary studies from the keyword-based search. The publications from the validation set and the known set of publications are also included. Lacerda et al. [30] is the only

tertiary study close to our topic. Therefore, we considered including the secondary studies from their tertiary study, which may be potentially relevant. We consider this as an acceptable trade-off to supplement our search results while not duplicating the efforts of Lacerda et al. [30]. Moreover, Lacerda et al. [30] only share one secondary study with the validation set, thus improving the chances of adding potentially relevant studies missed by the automated search. After removing the duplicated publications, we have 711 unique publications.

4.2. Selection results

Before applying topic specific selection steps, the first author identified publications out of the 711 search results in Fig. 2 that meet the selection criteria C0 and C1 in Table 1. Thus, publications with less than eight pages, not written in English or not published in a peer-reviewed venue, are excluded. A total of 163 publications that did not meet the page and language requirements were excluded leaving 548 secondary studies for the next phase of the selection. Among the 163 publications removed during this phase, four were non-English publications while the remaining were removed due to short length.

4.2.1. Pilot selection process

For the 12 secondary studies considered in the pilot selection, all authors agreed on seven out of twelve secondary studies while there were disagreements on the remaining five secondary studies, giving an initial agreement percentage of 58%. Only two out of six author-pairs had a substantial agreement, while two author-pairs had a moderate agreement. The disagreements were resolved in a discussion and the descriptions of the inclusion/exclusion criteria were further clarified.

4.2.2. Complete selection process

Next, each of the remaining 548 secondary studies were reviewed by two authors. While the first author applied the selection criteria to all secondary studies, 182 randomly selected secondary studies were assigned to the second, third, and fourth author, each. Table 2 used in [52], was utilized during the decision making process. A secondary study was excluded if it was resolved as “irrelevant” and it was included if it was agreed upon as “maybe” or “included” by both authors. In this round, the initial agreement among the author-pairs was 73%. The average of Cohen-Kappa inter-rater agreement between author-pairs was 0.64, which is substantial agreement [40,41].

The disagreements during this round were also resolved through discussion among authors and was utilized to further improve the clarity of the inclusion/exclusion criteria. After the study selection process, 135 secondary studies were included, while 413 secondary studies were excluded from the list of studies.

After the conclusion of the adaptive reading step, 99 secondary studies were retained while 36 secondary studies were identified as irrelevant to the topic. All secondary studies identified by the first author as irrelevant were reviewed by the second author see Table 6.

The first author read the full text of 99 papers. The full text for one secondary study [53] was not available (besides our best efforts), thus it was excluded. Two papers were identified as the same secondary studies [54,55] and the most recent of the two secondary studies [54] was retained. The first author read all the papers in this phase and further identified 37 secondary studies as irrelevant to the scope. The excluded secondary studies were reviewed by the second and fourth author and the authors agreed on excluding 35 secondary studies. Disagreements on the remaining two secondary studies were resolved through a discussion leading to inclusion of both, giving 62 secondary studies for quality assessment and data extraction.

As a reflection on the study selection process, we analysed how this final set of 62 secondary studies were initially assessed by authors after selection based on titles and abstract only in Table 6. Of those 62 secondary studies, four secondary studies were initially assessed as “Include-Exclude”, three secondary studies were initially assessed

Table 5
Strength of evidence criteria.

Strength of evidence	Secondary study DARE score	AQAS* score	PS in result	Reported external metrics	Number of “Yes” to criteria 6 to 12
Very High	At least one high quality secondary study	AQAS ≥ 10	PS ≥ 25	Yes	≥ 2 Yes
High	At least one high quality secondary study	AQAS < 10	PS ≥ 25	No	≥ 2 Yes
	At least one high quality secondary study	AQAS < 10	PS ≥ 25	No	< 2 Yes
	At least one high quality secondary study	AQAS < 10	$10 < PS < 25$	Yes	≥ 2 Yes
Moderate	At least one high quality secondary study	AQAS < 10	PS ≥ 25	No	< 2 Yes
	At least one high quality secondary study	AQAS < 10	$10 < PS < 25$	No	≥ 2 Yes
	At least one high quality secondary study	AQAS < 10	$10 < PS < 25$	Yes	< 2 Yes
	At least one high quality secondary study	AQAS < 10	PS ≤ 10	Yes	≥ 2 Yes
Low	At least one high quality secondary study	AQAS < 10	$10 < PS < 25$	No	< 2 Yes
	At least 1 high quality secondary study	AQAS < 10	PS ≤ 10	No	≥ 2 Yes
	One high or medium quality secondary study	AQAS < 10	PS ≤ 10	No	< 2 Yes
Very Low	One high or medium quality secondary study	AQAS < 10	PS ≤ 10	No	< 2 Yes

AQAS*: Additional quality assessment score.



Fig. 2. Selection process results (The count depicts included secondary studies at each stage).

Table 6
Results after preliminary selection.

S. No	Decision	Number of secondary studies	Comment	Included after full-text reading
1	Exclude–Exclude	320	Excluded	–
2	Exclude–Maybe	26	Review	0
3	Maybe–Maybe	5	Included	3
4	Include–Maybe	37	Included	14
5	Include–Include	78	Included	41
6	Include–Exclude	82	Review	4
–	Studies in preliminary selection	548	–	–

as “Maybe–Maybe” and 14 secondary studies were initially assessed as “Include–Maybe”. Since a large number of secondary studies might have been excluded solely based on the decision rules mentioned in Table 2, we recommend that authors should include secondary studies categorized as “Include–Maybe” and “Maybe–Maybe” for full-text reading as a large percentage of these secondary studies might be relevant. The authors should also carefully review secondary studies marked as “Include–Exclude” list as immediate exclusion may result in the exclusion of relevant secondary studies.

4.3. Data extraction

4.3.1. Piloting of data extraction

In the piloting of the data extraction form, the authors agreed on 76% of the data extracted for one randomly selected paper. The differences were discussed and resolved.

4.3.2. Data extraction validation

In a post hoc validation of the extracted data, the third author independently did data extraction for six secondary studies. We found that all entries related to demographic data, internal/external attributes, reported metrics matched between the first and third authors. However, minor differences were observed in two fields, namely, recommendations/findings and synthesis method utilized. The differences were discussed and corrective action was taken to resolve them.

4.4. Quality assessment of the secondary studies

In the pilot round of quality assessment, there was agreement on the first and fifth questions (see Table 19) related to the inclusion and exclusion criteria and synthesis of secondary studies. The authors’ results did not match questions two, three, and four regarding sufficient search coverage, quality assessment, and secondary study description. After discussing the misalignment in understanding, the differences were resolved.

The validation of a 10% randomly selected sample of secondary studies showed that there was more than 80% agreement on the first two DARE questions, 66% agreement on question 3, and 50% agreement on the last two questions. A meeting was held to discuss the differences and improve the alignment between the authors. It was observed that all disagreements were minor, e.g., one author assigned “yes” while the other author assigned “partly”, or when one author has assigned “partly” the other author assigned “no” when answering questions. Based on the improved understanding, the first author reviewed all papers assigned as “yes” for Questions 4 and 5 (see Table 19) of the quality criteria. The fourth author reviewed the updated results, and all changes were agreed upon.

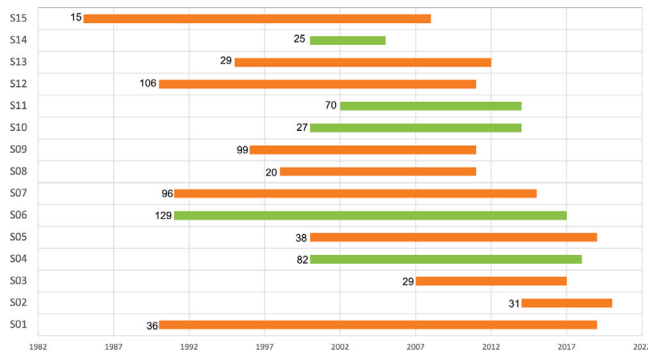


Fig. 3. Number of primary studies and years covered by the included secondary studies.

After removing secondary studies with scores less than or equal to 1.5, 55 secondary studies remained. As DARE is not designed to evaluate the quality of multi-vocal reviews, quality assessment-based selection was not applied to MLRs. The secondary studies removed due to low DARE score are listed online.³ Detailed results of quality assessment are reported in Section 5.1.5 as part of the characteristics of the included secondary studies.

4.5. Secondary studies providing links between external quality attributes and source code metrics

From the list of 55 secondary studies, only 15 secondary studies provide evidence regarding a link between external quality attributes and source code metrics (see Fig. 2). These secondary studies either provide quantitative (e.g., correlation or prediction results) or qualitative evidence (e.g., stated claims or comments on source code metrics as good or poor indicators) of the link between source code metrics and external quality attributes. Forty secondary studies³ only report the *usage* of source code metrics for assessing or measuring external quality attributes. These 40 secondary studies have not commented on how well they measure external quality attributes, nor did they provide any qualitative or quantitative evidence exploring or explaining the link between the source code measures and the external quality attributes. While usage alone may be considered evidence, we focus on explicitly stated evidence in this tertiary study. We have thus identified 15 secondary studies reporting such evidence.

4.6. Removal of double-counting results in secondary studies

We resolved overlaps in primary studies reported by the secondary studies by counting duplicate primary studies only once. This was possible for the case where we could trace the evidence regarding the link between source code metrics and external quality attributes reported in the secondary study to the primary study included. We mitigated overlaps in primary studies between S06 and S12 (seven primary studies), S01 and S04 (nine primary studies), S12 and S14 (one primary study), and S06 and S14 (one primary study).

5. Results and analysis

5.1. RQ1: Characteristics of secondary studies

The 15 selected secondary studies are shown in Table 18. Ten of them are systematic literature reviews (SLRs) and five systematic mapping studies (SMSs). The earliest secondary study that fulfilled our selection criteria was published in 2009. The search was performed in February 2021, which may explain why no secondary studies from 2021 are included in our sample. The researcher's interest in software code metrics and reporting relationship with quality attributes has been

steady over the years, with nine out of 15 of the secondary studies published between the years 2016–20.

The years covered by the secondary studies are shown in Fig. 3. Half of the published articles cover the years between 1998 and 2014. In their search results, only three secondary studies had included the years prior to 1991 (when Chidamber and Kemerer metrics suite was introduced), indicating that researchers have focused on primary studies conducted after this year.

5.1.1. Secondary study aims

We have analysed the research questions and stated aims from the secondary studies based on Fenton's classification of internal and external quality attributes [9] (see also Fig. 1). Furthermore, we classify the secondary studies in each sub-topic based on metrics utilization, i.e., whether the source code metrics have been utilized for exploring the relationship between source code metrics and external quality attributes (i.e., correlation studies) or explaining the relationship between source code metrics and external quality attributes (i.e., prediction studies).

Maintainability is the most frequently studied external attribute, with six secondary studies investigate maintainability prediction while five secondary studies investigate reliability prediction as depicted in Fig. 4. One secondary study (though reports evidence for reliability) aimed to report influential metrics and their aggregation without specifying the internal or external attributes, which we have classified as "Unclear". Similarly, one secondary study investigates the internal attributes of code categorized into coupling and cohesion.

5.1.2. Systems studied

We classified the systems studied as open-source, industrial systems, or academic systems. The number of systems studied is shown in Fig. 5. Of the 15 secondary studies, five did not mention the software systems used in their primary studies.

Open-source systems were the most popular type of software systems studied and were reported by nine out of 15 secondary studies, while seven out of 15 secondary studies utilized industrial systems. Academic systems were reported by six out of 15 secondary studies. Four secondary studies use systems from all three categories, while only two out of 15, secondary studies only use one type of system. Sixty-five unique open-source systems were reported among the included secondary studies. JEdit.⁵ is the most often reported open-source system used by three secondary studies followed by Mozilla⁶ and Apache⁷. Among the secondary studies that used public datasets, the PROMISE⁸ and NASA⁹ datasets have been reported by more than three secondary studies.

5.1.3. Databases used

The databases used to perform an automated search by the included secondary studies are depicted in Table 7. The most frequently used databases for search in the included secondary studies are IEEE Xplore, ACM Digital Library, Elsevier, and Scopus. Of the 15 secondary studies, 11 searched in IEEE Xplore and ACM Digital library while nine selected Elsevier and Scopus. 11 out of 15 secondary studies searched in four or more databases. The highest number of databases searched by a secondary study was nine, reported in two secondary studies. Four secondary studies, each, searched in four databases, which is the most common choice among the selected studies and is also recommended by systematic study guidelines [31,45]. Among the included secondary studies, three secondary studies searched in two or less than two

⁵ <http://www.jedit.org/>

⁶ <https://www.mozilla.org/en-US/>

⁷ <https://www.apache.org/>

⁸ <http://promise.site.uottawa.ca/SERepository/>

⁹ <https://data.nasa.gov/>

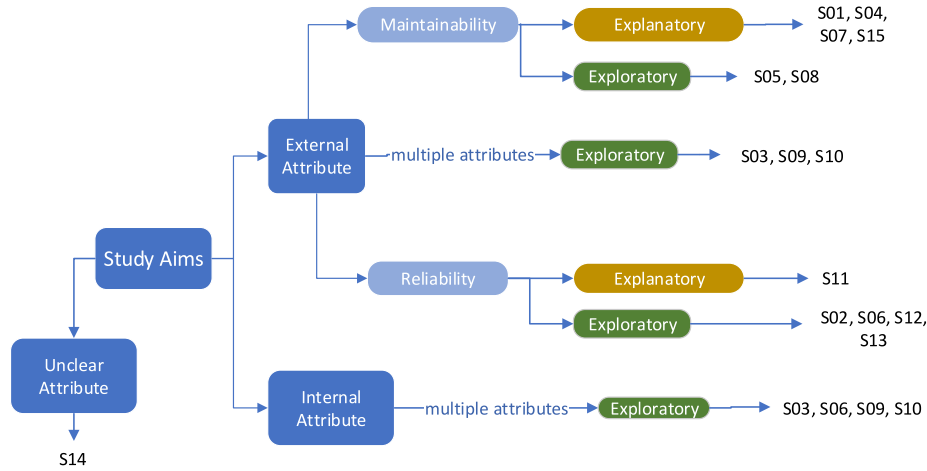


Fig. 4. Secondary study aims.

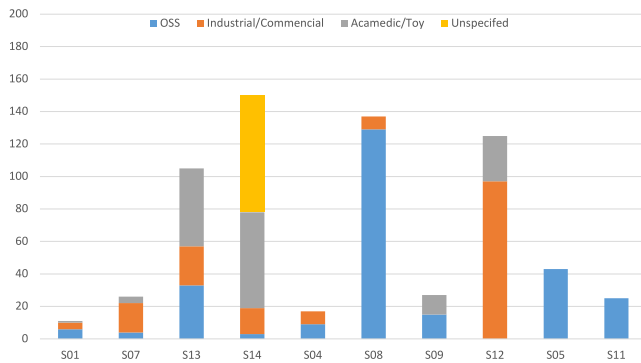


Fig. 5. Type of systems studied.

Table 7
Databases used for automated search by secondary studies.

Databases name	Studies
ACM Digital Library	S01, S03, S04, S05, S06, S07, S09, S10, S11, S12, S15
EI Compendex	S07, S09, S12
Google Scholar	S02, S04, S06, S07, S10, S13
IEEE Xplore	S01, S02, S04, S05, S06, S07, S09, S10, S11, S12, S15
Inspec	S09, S12, S15
Others	S01, S04, S07, S11, S12, S13
ScienceDirect – Elsevier	S01, S02, S03, S04, S06, S07, S10, S12, S15
Scopus	S01, S04, S05, S07, S09, S11, S13, S14, S15
Springer Link	S01, S02, S04, S05, S07, S12, S15
Wiley Online Library	S01, S05, S07

Table 8
Quality assessment of secondary studies.

Type	ID	Total	AQAS*	Q1	Q2	Q3	Q4	Q5
SLR	S02	2.5	0	1	0.5	0	0.5	0.5
	S08	2.5	0	1	0	0	1	0.5
	S03	3.5	2	1	0	1	1	0.5
	S07	3.5	8	1	1	0.5	0.5	0.5
	S01	4	10	1	1	1	0.5	0.5
	S12	4	7	1	1	1	0.5	0.5
	S13	4	5	1	0.5	1	1	0.5
	S15	4	0	1	1	1	0.5	0.5
	S05	4.5	7	1	0.5	1	1	1
	S09	4.5	7	1	0.5	1	1	1
SMS	S06	2	0	1	0.5	0	0.5	0
	S11	2	0	0.5	0.5	0.5	0.5	0
	S14	2	0	1	0	0	0.5	0.5
	S04	4	4	1	1	1	0.5	0.5
	S10	4.5	2	1	1	1	1	0.5

AQAS*: Additional quality assessment score.

source code metrics with external quality with focus on aspect-oriented, functional or declarative programming paradigms. This highlights an important research gap for future researchers in source code metrics. Despite the recent interest in feature-oriented development practices, only one secondary study investigated feature-oriented programming paradigm.

5.1.5. Quality assessment results

After applying our quality assessment criteria mentioned in Table 19, quality assessment results for the secondary studies are reported in Table 8. The range of total scores for a secondary study is 0–5. Ten studies have scored above 3.5 thus are classified as high quality secondary studies according to Curcio et al. [47] while five secondary studies are medium quality secondary studies. We excluded seven secondary studies³ during study selection due to a quality score equal to or below 1.5, see Section 4.4. Interestingly, two out of ten SLRs do not report any quality assessment of the included primary studies. In contrast, three out of five mapping studies perform a quality assessment.

We categorized the quality assessment questions that the included secondary studies used to evaluate the quality of their included primary studies. We categorized these into 18 quality assessment questions shown in Table 9. Seven secondary studies assessed whether the included primary studies clearly stated their objectives, study limitations, and validity threats. While, four secondary studies assessed if the primary studies justified the prediction methods used. For each secondary study, we calculated an additional quality assessment score

databases, thus not fulfilling the sufficient search criteria of DARE as described by Budgen et al. [45]. Six out of 15 secondary studies searched in either three or four databases without incorporating an extra search strategy or have performed searches in a restricted set of venues.

5.1.4. Reported programming paradigms

The programming paradigms reported by each secondary study are categorized in Table 10. The programming contexts observed include object-oriented (14 secondary studies), procedural languages (8), and feature-oriented (1). Of the 14 secondary studies investigating the object-oriented paradigm, six focused solely on an object-oriented programming context. None of the secondary studies investigated links of

Table 9

Categorized quality assessment questions from included secondary studies (#SS denotes the number of included secondary studies).

S. No	Quality assessment question	#SS
QA1	Whether the objectives and scope of the study are stated clearly?	7
QA2	Does the study provide relevant literature?	2
QA3	Do the research questions, purposes, or hypotheses logically flow from the introductory material?	3
QA4	Are the research questions addressed clearly?	2
QA5	Does the study have a sufficient number of citations?	2
QA6	Are the results of the study reported in a clear manner?	5
QA7	Is the programming language stated?	1
QA8	Was an analysis conducted to check for outliers in the data? (if unclear enter No)	1
QA9	Are negative findings presented?	2
QA10	Does the study discuss how the results add to the literature?	1
QA11	Does the study provide a clear description of the external quality attribute being investigated?	2
QA12	Does the study justify the prediction method used?	4
QA13	If a study deals with more than one prediction technique is the comparative analysis conducted?	3
QA14	Is there a description of limitations and threats to the validity of research?	7
QA15	Are the data analysis techniques clearly defined and described?	2
QA16	Are the predictors effectively chosen using feature selection/dimensionality reduction techniques?	1
QA17	Did the study perform statistical hypothesis testing?	2
QA18	Whether the independent variables were clearly defined?	3

Table 10

Reported programming paradigms.

Paradigm	Studies
Object Oriented	S01, S02, S03, S04, S05, S06, S07, S08, S09, S11, S12, S13, S14, S15
Procedural	S01, S02, S03, S04, S07, S11, S12, S15
Feature Oriented	S03

beside the DARE score shown in Table 8. Eight of the 15 secondary studies considered whether their primary studies provided a detailed methodology, whether the data collection steps were clearly described, and the reproducibility of results by evaluating whether public datasets were used.

5.2. RQ2: Strength of evidence of link between source code metrics, internal quality attributes, external metrics and external quality attributes

This section presents the strength of evidence of the links between source code metrics (described online³) and external quality attributes as reported in the secondary studies. The strength of evidence is based on several factors, e.g., quality score of the secondary study, number of primary studies reporting the evidence, status of validation of metrics as reported by secondary study (see Section 3.5.4). The scheme used to interpret evidence reported in secondary studies is described in Table 4 and strength of evidence is classified according to Table 5. The reported evidence in the included secondary studies relates to maintainability, reliability, and security attributes and their external metrics as described in Fig. 1.

5.2.1. Strength of evidence on maintainability

Nine studies report evidence on the link between source code metrics, external metrics (see Table 20), and maintainability. Table 11 depicts results from exploratory studies along with quality score, our interpretation of the reported relationship and strength of evidence. Results from explanatory studies are reported in Table 12. Of these nine secondary studies, two studies have medium quality while seven are of high-quality.

In the exploratory secondary studies (see Section 3.5.2), four studies (S03, S06, S08, and S09) reported correlation levels between source

code metrics and maintainability. Among these, only two studies (S08, S09) reported statistical significance levels (p-values) for their results. One secondary study (S05) reported only claims without stating evidence. Change is the most often reported external metric used by two studies while one secondary study has maintenance effort. Another secondary study differentiates between changes and change-proneness, defined as the likelihood of change. Three secondary studies (S03, S09 and S15) utilize several external metrics and present aggregated results for external metrics. Thus, it is not possible to report individual external metrics used to link source code metrics with external quality attributes for these three secondary studies. Complexity is the most investigated internal quality attribute when linking source code metrics with maintainability.

From Table 11, exploratory studies on maintainability have varying strength of evidence ranging from moderate to very low. CBO and RFC are reported by two secondary studies to be significantly linked with maintainability with consistent moderate strength of evidence from more than 20 primary studies. Similarly, strong evidence suggests that inheritance measures NOC and DIT have insignificant relation with maintainability.

Similarly, WMC-McCabe, LCOM2 and LOC show significant links with maintainability. Interestingly, DIT has a weak positive link with maintainability when external measure is either change or effort related. Several other measures show a significant link with maintainability, including NOM, ICH, MPC, LCOM5 and DAC, though the strength of evidence for these relationships is very low.

Four explanatory studies report quantitative results linking source code metrics with maintainability. In Table 12, we report our interpretation of reported results (based on criteria mentioned in Section 3.5.2) linking source code metrics suites and prediction model used to explain the relationship with external metric of maintainability. While all four secondary studies achieve high DARE scores in our assessment, the overall strength of evidence of link between source code metrics and maintainability in explanatory studies is “Low” and “very low” since less than ten primary studies report evidence of the link.

Three accuracy measures are reported in the secondary studies: Mean Magnitude Relative Error (MMRE) or Mean Absolute Relative Error (MARE), Absolute Relative Error (ARE), and Ratio of true positive.¹⁰ CK metrics and Li & Henry metrics are the most commonly used sets of metrics in explanatory studies and are used by three secondary studies (S01, S04, S07). The combination of CK and Li & Henry metric suites used along with prediction models are largely classified as “Unclear” for sixteen different prediction techniques. One possible explanation could be the varying effectiveness of prediction models that they used in the secondary studies and the dependency on the datasets used.

5.2.2. Strength of evidence on reliability

Eight secondary studies report strength of evidence on the link between source code metrics, external metrics and reliability. Table 16 depicts results from exploratory studies along with quality score, our interpretation of the reported relationship and strength of evidence. Table 17 reports results from explanatory studies on reliability. Four out of the eight secondary studies have a medium DARE score while the other four secondary studies have high DARE score. Similar to the previous section, complexity is the most linked internal attribute to reliability. Among the external measures reported, fault-proneness and number of defects are the most associated measures.

In the exploratory studies, seven secondary studies report correlation results for the link between source code metrics and reliability. Among these secondary studies, only one secondary study (S09) reported confidence intervals for their results. The other secondary studies reported the correlation results without providing confidence

¹⁰ We contend that MMRE and MARE are both summary statistics based on taking numerical average of a set of Absolute Relative Errors (ARE).

Table 11
Strength of evidence: maintainability linked to metrics in exploratory studies.

External metrics	Internal attribute	Source code metric	Secondary studies	Overlap in PS	DARE score	Relationship indication					Strength of evidence
						“++”	“+”	“0”	Unclear	Sig*	
Not reported in SS	Coupling	CBO	S03V, S09	No	H, H	–	–	2	9	20	Moderate
	Inheritance	DIT	S03V, S09	No	H, H	–	–	24	8	6	Moderate
	Inheritance	NOC	S03V, S09	No	H, H	–	–	34	8	5	Moderate
	Complexity	RFC	S03V, S09	No	H, H	–	–	3	8	26	Moderate
	Complexity	WMC-McCabe	S09	–	H	–	–	1	8	28	Moderate
	Size	LOC	S09	–	H	–	–	–	–	28	Moderate
	Cohesion	LCOM2	S09	–	H	–	–	–	8	13	Moderate
	Size	NC	S09	–	H	–	–	–	1	1	Moderate
Change-proneness	Coupling	CBO	S05	–	H	–	–	–	–	11	Moderate
	Inheritance	DIT	S05	–	H	–	–	–	–	3	Low
	Cohesion	LCOM	S05	–	H	–	–	–	–	5	Low
	Complexity	RFC	S05	–	H	–	–	–	–	8	Low
	Size	SLOC	S05	–	H	–	–	–	–	14	Moderate
	Complexity	WMC	S05	–	H	–	–	–	–	10	Moderate
Change	Coupling	CBO	S06V, S08	No	M, M	2	1	1	–	–	Low
	Inheritance	NOC	S06V, S08	No	M, M	–	1	2	1	–	Low
Maintenance Effort	Coupling	CBO	S06V	–	M	–	–	–	1	–	Low
	Inheritance	DIT	S06V	–	M	–	1	–	–	–	Low
	Cohesion	LCOM	S06V	–	M	–	1	–	–	–	Low
	Inheritance	NOC	S06V	–	M	–	1	–	–	–	Low
	Complexity	RFC	S06V	–	M	–	1	–	–	–	Low
	Complexity	WMC	S06V	–	M	–	1	–	–	–	Low
Not reported in SS	Cohesion	CAMC	S09	–	H	–	–	–	–	3	Very Low
	Stability	CoV	S03V	–	H	–	1	–	–	–	Very Low
	Complexity	DAC	S09	–	H	–	–	1	1	7	Very Low
	Inheritance	DDC	S03V	–	H	–	1	–	–	–	Very Low
	Coupling	EC	S09	–	H	–	–	1	1	–	Very Low
	–	GoS	S03V	–	H	–	1	–	–	–	Very Low
	Cohesion	ICH	S09	–	H	–	–	–	–	9	Very Low
	–	LoS	S03V	–	H	–	1	–	–	–	Very Low
	Cohesion	LCOM1	S09	–	H	–	–	3	–	6	Very Low
	Cohesion	LCOM5	S09	–	H	–	–	1	–	6	Very Low
	Coupling	MPC	S09	–	H	–	–	1	–	7	Very Low
	Inheritance	NMO	S09	–	H	–	–	4	–	4	Very Low
	Complexity	NOM	S09	–	H	–	–	–	1	7	Very Low
	Cohesion	TCC	S09	–	H	–	–	3	–	5	Very Low
	Complexity	WMC	S03V	–	M	–	–	1	–	–	Very Low
Change	Inheritance	DIT	S08	–	M	2	–	1	–	–	Very Low
	Cohesion	LCOM	S08	–	M	–	2	1	–	–	Very Low
	Complexity	RFC	S08	–	M	–	1	2	–	–	Very Low
	Size	LOC	S08	–	M	–	2	–	–	–	Very Low
	Complexity	WMC	S08	–	M	1	–	1	–	–	Very Low

Sig*: Results do not distinguish between positive or negative correlation when reporting significantly linked source code metrics.

intervals. Two secondary studies S02 and S14 state claims without giving any evidence of results. Fault-proneness is the most utilized external metric for reliability. S09 reports multiple external metrics and thus is not possible to report individual external metric used to link source code metrics with reliability.

Exploratory studies on reliability have varied strength of evidence ranging from “High” to “Very low” (due to space limitations, results graded as “Very low” are only included in the online supplement³). CBO, RFC, LOC and WMC show positive links with reliability with consistently high strengths of evidence from two high quality secondary studies and more than 40 primary studies. At the same time, strong evidence suggests that LCOM, DIT, and NOC have insignificant relationship with reliability.

Among the secondary studies with moderate strength of evidence, several metrics including AMC, WMC-McCabe, OCAIC, LCOM2, cyclomatic complexity and LOC have been significantly linked with reliability. However, TCC and NOC, are shown to be insignificantly correlated with reliability. Thus, we recommend that TCC and NOC may not be used when assessing reliability. One secondary study of medium quality only provides strength of evidence of a link between code metrics, error-proneness, and bug-prediction which may be considered weak compared to the results from high quality studies.

Only one explanatory secondary study, S11, reports accuracy measures as evidence of the relationship between source code metrics and the number of faults as the external metric for reliability in Table 17. S11 reports several different accuracy measures to describe the link between metrics and reliability. Due to the medium DARE score and fewer primary studies reporting results, the overall quantitative evidence is assigned as “low”.

5.2.3. Strength of evidence on security

One secondary study S03 with a high DARE score reports seven metrics correlated with security. The relationship is summarized in Table 14. Four metrics are positively linked with vulnerabilities as the sub-attribute of security. The description of these metrics is available online.³ This seems to be an intuitive result as software code with several external configuration options may be easier to compromise or changing the externally configurable options may lower a module's security settings.

5.3. RQ3: Efficacy of prediction models

Seven out of 15 secondary studies (S01, S02, S04, S05, S07, S11 and S15) reported models used to predict external quality attributes

Table 12

Strength of evidence: maintainability linked to source code metrics in explanatory studies.

External metrics	Source code metrics	Prediction model	Secondary studies	DARE Score	Relationship indicator		Strength of evidence
					“+”	Unclear	
Change	Halstead metrics, LCOM, DAC & misc metrics	SFs	S15	H	1	–	Very Low
		SFSf	S15	H	1	–	Very Low
	CK & LH & Li suite	ANFIS	S07	H	1	–	Low
	CK & LH suite	GRNN	S01	H	–	9	Low
		ANN	S01	H	–	3	Low
		FFBN	S01	H	–	3	Low
		MLP	S01	H	–	4	Low
		PNN	S01	H	–	7	Low
		KN	S01	H	–	7	Low
		GMDH	S01	H	–	9	Low
		RT	S01	H	–	5	Low
		M5P	S01	H	–	4	Low
		BN	S01	H	–	4	Low
		SVM	S01	H	–	9	Low
		Kstar	S01	H	–	9	Low
		ELM	S04	H	–	1	Low
		MLP	S04	H	–	1	Low
		SVM	S04	H	–	1	Low
		Neuro-GA	S04	H	–	1	Low
		Neuro-Fuzzy	S04	H	–	1	Low
		GMDH	S04	H	1	–	Low
		GA	S04	H	1	–	Low
		PNN	S04	H	1	–	Low
		ANN	S07	H	–	1	Low
	CK & Tang suite	GRNN	S07	H	–	1	Low
	CK Suite	ANN	S07	H	–	1	Low
		BPN	S07	H	–	1	Low
		PNN	S07	H	1	–	Low
Maintainability Index	Halstead 'E', Halstead 'V'	Polynomial model	S15	H	1	–	Very Low
	OSAVG, CSO, CSA, SNOC	Univariate Linear Regression Analysis	S15	H	1	–	Very Low

Table 13

Metrics that are good indicators of external quality attributes.

External attribute	Internal attribute	Source code metrics	Secondary studies	Relationship indicator			Strength of evidence
				“++”	“+”	Sig*	
Maintainability	Cohesion	LCOM2	S09	–	–	13	Moderate
	Complexity	RFC	S03V, S09	–	–	26	Moderate
		WMC-McCabe	S09	–	–	28	Moderate
	Coupling	CBO	S03V, S09	–	–	20	Moderate
	Size	LOC	S09	–	–	28	Moderate
Reliability	Code Churn	Changes	S12	16	–	–	Moderate
		Churn	S12	15	–	–	Moderate
		Age	S12	10	–	–	Moderate
	Cohesion	LCOM	S13V	–	–	15	Moderate
		LCOM5	S09	–	–	12	Moderate
		LCOM2	S09	–	–	21	Moderate
	Complexity	RFC	S06V, S12	51	–	5	High
		WMC	S06V, S12	44	–	4	High
		Cyclomatic Complexity	S12	–	43	–	High
		RFC	S13V	–	–	21	Moderate
		VG (McCabe)	S09	–	–	32	Moderate
		WMC	S13V	–	–	21	Moderate
		AMC	S09	–	–	35	Moderate
		WMC-McCabe	S09	–	–	62	Moderate
		NOM	S09	–	–	16	Moderate
		RFC	S09	–	–	50	Moderate
	Coupling	CBO	S06V, S12	48	–	4	High
		OMMIC	S09	–	–	10	Moderate
		CBO	S13V	–	–	23	Moderate
		OCAIC	S09	–	–	10	Moderate
		CBO	S09	–	–	60	Moderate
	Maturity	Past Faults	S12	–	10	–	Moderate
	Size	LOC	S12, S14V	–	61	–	High
		SLOC	S13V	–	–	16	Moderate
		LOC	S09	–	–	58	Moderate

Table 14

Strength of evidence: security linked with source code metrics in exploratory studies.

Ext sub-attribute	Internal attribute	Source code metric	Secondary study	PS in Overlap	DARE score	Relationship indicator			Strength of evidence
						“+”	“0”	“-”	
Vulnerability	Complexity	Degree Centrality (outgoing)	S03 ^V	–	H	1	–	–	Low
Vulnerability	Complexity	Eigenvector Centrality	S03 ^V	–	H	1	–	–	Low
Vulnerability	Complexity	Internal Configuration Options	S03 ^V	–	H	1	–	–	Low
Vulnerability	Complexity	Number of Internal #ifDefs	S03 ^V	–	H	1	–	–	Low
Vulnerability	Complexity	Betweenness Centrality	S03 ^V	–	H	–	1	–	Low
Vulnerability	Complexity	Degree Centrality (ingoing)	S03 ^V	–	H	–	1	–	Low
Vulnerability	Complexity	External Configurations Options	S03 ^V	–	H	–	–	1	Low

using source code metrics. Among these, two secondary studies S11 and S15 do not compare their effectiveness in terms of prediction accuracy due to the different focus of the secondary study. Among the studies that compare different prediction techniques (for description see online³), only S02 reports the comparison for reliability, while other secondary studies report a comparison for maintainability prediction models. The results of the comparisons are shown in Table 15. In our sample, PNN, GA, ANFIS, and GMDH are superior prediction techniques for maintainability.

S01 compares the effectiveness of statistical models with machine learning based models using UIMS and QUES data-sets along with proprietary software and open source system. The secondary study notes that machine learning based techniques outperform statistical models based on linear regression and multiple linear regression. Among the machine learning techniques, Group Method of Data Handling (GMDH) based models outperform other machine learning techniques including General Regression Neural Network (GRNN), Feed Forward Back propagation Network (FFBN), Probabilistic Neural Network (PNN), KStar, Kohan Network (KN) and Support Vector Machine (SVM). S01 further observes that the effectiveness of hybrid techniques that combine statistical and machine learning models is inconclusive.

One secondary study, S07, also reported machine learning algorithms to be superior to statistical methods. Based on the results, Adaptive Neuro Fuzzy Inference System (ANFIS) based models perform slightly better than Artificial Neural Network (ANN) and other methods reported Back Propagation Network (BPN), PNN, and GRNN.

Another secondary study, S04, investigated machine learning based prediction models and point out that the effectiveness of models is dependent on the data-set used as well as the independent variables utilized. Among the secondary studies, changing the data-set revealed different techniques to be superior. In the secondary studies that use UIMS and QUES datasets, GMDH is the superior technique with better MMRE results compared to Genetic Algorithm (GA) and PNN. Among the secondary studies that only used QUES dataset, Kstar and Mamdani-based Fuzzy Logic (MFL) perform better than other methods.

Similarly, S05 notes that search-based machine learning algorithms such as Bagging (BG) have better mean accuracy than SVM and k-means. In terms of the mean area under the curve (AUC), BG, random forests (RF), and adaptive boosting (AB) techniques are suggested to be superior.

S02 notes there is no explicit agreement on the best prediction technique for reliability among the six different prediction techniques used. The compared prediction techniques are Bayesian Network (BN), Neural Network (NN), Support Vector Machine (SVM), Clustering, Feature Selection (FS), and Ensemble Learning (EL).

When comparing the prediction accuracy of ANFIS from S04 and GMDH, GA and PNN results from S07 reporting the same combination of metrics, GMDH achieves lower MMRE values in all the four modelling techniques, thus can be suggested to be more suited for prediction studies. However, when comparing different algorithms based on the varied results in our analysed secondary studies, we can suggest that the effectiveness of prediction techniques is context-dependent and data-sensitive, and more analysis is needed in evaluating and comparing the different techniques using similar datasets and independent variables. Given the large variety of machine learning techniques, statistical methods, nature-inspired techniques and hybrid methods, this can be an interesting dimension for future analysis.

Table 15

Comparison of prediction models reported.

Superior model	Models compared	Secondary study
GMDH	GRNN, FFBN, PNN, Kstar, KNN, SVM	S01
ANFIS	ANN, BPN, PNN, GRNN	S07
GMDH	GA, PNN	S04
BG, RF	SVM, K-means	S05
None	BN, NN, FS, SVM, EL, Clustering	S02

6. Discussion

The focus of our tertiary study is to provide a comprehensive evaluation of research on source code quality by synthesizing and categorizing the state-of-the-art literature that reports evidence-based links between source code metrics and external quality attributes in secondary studies. In this section, we reflect on our results and analyses in the light of related literature.

Out of 55 secondary studies using source code metrics to study external attributes, only 15 secondary studies of sufficient quality reported evidence on the relationship between source code metrics and external quality attributes. To better understand the relationship between source code metrics and external quality attributes, more secondary studies are needed that synthesize available empirical evidence (e.g., meta-analyses). The results of the present tertiary study provides a starting point and provides information about source code metrics that show consistent relationships with two external quality attributes; maintainability and reliability.

6.1. External quality attributes

A small subset of quality attributes, namely, maintainability, reliability, and security from ISO-IEC 25010 [6] have been reported with link of evidence with source code metrics in our included secondary studies. All 15 secondary studies in our sample have focused on reliability or maintainability, while security has received considerably less focus (one secondary study). We have not found evidence linking source code metrics with functional suitability, compatibility, usability and other external quality attributes outlined in Fig. 1. Colakoglu et al. [56] also observe maintainability and reliability to be the most studied external quality attributes. Intuitively, we may argue that other external quality attributes such as usability are less related to source code metrics and more to the user interface or the ease of interaction. Thus, we have no secondary studies linking source code with usability in our included studies.

However, in other secondary studies considered during the study selection process, source code metrics have been linked with functional correctness¹¹ [57]. While highly relevant to our scope, this secondary

¹¹ The secondary study defined functional correctness as “the degree to which a product or system provides the correct results with the needed degree of precision” and quantified functional correctness in terms of *fault count*, *fault proneness*, and *fault density*.

study was excluded due to a low DARE score. We suggest conducting high-quality secondary studies on evaluating the link between source code metrics and other external quality attributes by extending secondary studies such as [57,58].

6.2. Internal quality attributes

Complexity, coupling and size are most popular among internal quality attributes and have consistently predicted or assessed maintainability. Our results are in agreement with Arvanitou et al. [8] who also report complexity, cohesion, and coupling to be the most frequently studied internal attributes. We also note the lack of evidence for other internal attributes such as polymorphism, abstraction, code churn, encapsulation, and hierarchies. Arvanitou et al. [8] also observe abstraction to be a less studied attribute. While polymorphism has not been linked to any external quality attributes in our included secondary studies, Briand et al. [14] report polymorphism to be positively linked to reliability. Our results show that inheritance has a weak positive link with maintainability. One possible explanation for this could be that software systems used to study the link between DIT and maintainability have little less variation in class hierarchies. Our results agree with Saxena et al. [59], who also noted that inheritance metrics, namely, DIT and NOC, were noted to have inconsistent results for fault-proneness. Catal et al. [60] noted that complexity metrics evaluated at the method level have been predominantly used for fault prediction. We observed a similar trend in our results, where complexity metrics from Halstead and McCabe have been evaluated with multiple prediction models for fault prediction.

6.3. External metrics

Among the secondary studies that reported a description of the external metric, change is described as “Lines of code added, deleted, and modified” in S01. In contrast, S04 described it as “The number of changes made to the source code (changes in LOC, change in modules, change in class)”. Since adding comments to a source code may also result in changes in LOC, there is a minor difference between the two descriptions of the external metric. Similarly, S06 describes fault-proneness as “The probability of detecting faulty classes”, whereas S14 describes it as “The probability of exhibiting a fault” without specifying the scope of the evaluation.

Since the maintainability index [61] uses complexity and size metrics to interpret maintainability, it is intuitively correlated to certain source code metrics. Thus results from prediction models that use it as a dependent variable may be misleading. However, only one of the earliest secondary study (S15) reports using the maintainability index as the external metric; we still feel that the results in the tertiary study can be relied upon and utilized. This shows that in recent years, researchers have preferred to use more accurate external metrics instead of the maintainability index.

While the accuracy of prediction of bugs and defects is an important element in assessing software product quality, the severity of bugs and defects also impacts software quality. In our tertiary study, we have only found evidence of the effectiveness of prediction. As suggested in S02, using prediction capability and bug severity into account gives a better view of software product quality.

6.4. Good indicators for external attributes

In this section, we synthesize the results from the previous Sections 5.2.1 and 5.2.2. We synthesized the results with “High” or “Moderate” strength of evidence and have been significantly linked with quality attributes in exploratory and explanatory studies in Tables 11, 12, 14, 16, and 17. Table 13 shows several metrics to be significantly linked with reliability and maintainability. In our included secondary

studies, there are no high-quality secondary studies that report the link between source code metrics and security.

Based on the results, we can suggest using this selective set of metrics over others as they may be better indicators of related external attributes. We note that certain source code metrics related to cohesion, complexity, coupling, and size can be observed to be good indicators of maintainability with consistent strength of evidence from two high-quality secondary studies reporting results from more than ten primary studies.

The strength of evidence for reliability is considerably richer as compared to maintainability. Source code metrics related to code churn, cohesion, complexity, coupling, maturity, and size are good indicators of reliability with consistent strength of evidence from four high-quality secondary studies reporting results from more than ten primary studies.

It is interesting to note that a similar set of source code metrics are suggested to be good indicators of both maintainability and reliability. These source code metrics do not belong to any particular set of metric suites. Individual source code metrics from widely popular CK, QMOOD, and Li & Henry metric suites have been shown to have varied reported evidence. Inheritance metrics DIT and NOC from CK metric suites have been shown to be poorly linked with maintainability and reliability. Variants of LCOM such as LCOM2 and LCOM5 have been noted to be more effective indicators of cohesion attributes. Complexity metrics can be considered as consistent indicators of quality attributes as we note that complexity metrics have shown strong strength of evidence for both maintainability and reliability. Code churn and past faults are also reported as good indicators for reliability but have not shown a similar link with maintainability. Also, in our results, there are no source code metrics that have a predominantly negative relation with maintainability and reliability.

6.5. Beyond object-oriented & static metrics

Most of the evidence available in the included studies links source code metrics with external attributes in the object-oriented and procedural programming paradigms. Only one secondary study investigated feature-oriented programming, while none of the secondary studies focused on aspect-oriented, functional or declarative programming. This limits the applicability of our results to other programming paradigms, which need to be evaluated further.

Another interesting trend is that all of the evidence presented in the previous sections is from static source code metrics. Dynamic source code metrics provide useful information regarding the software’s runtime characteristics and software quality during execution. Our analysis has not reported empirical evidence of the link between dynamic source code metrics and external attributes. As suggested by Malhotra et al. in S01 and S07, the combined use of static and dynamic source code metrics to assess quality attributes may aid the source code community in improving the assessment of quality attributes using source code metrics. Tahir et al. [62] have investigated challenges in the selection and implementation of dynamic metrics, which may explain why dynamic metrics have been used less frequently. Several studies have investigated the effectiveness of using in-process metrics collected during testing [63,64] to predict post-release software quality with promising results. The included secondary studies have not considered using in-process metrics during testing. In-process metrics may be good indicators of external quality attributes during testing compared to internal code quality metrics.

In S12, Radjenović et al. point out that in highly iterative development environments with frequent code changes, object-oriented metrics are noted to be less effective in assessing external quality attributes. In contrast to product metrics, process metrics are recommended instead of object-oriented metrics and are considered more suitable for highly iterative development environments. Malhotra et al. (S05) also recommend using process metrics in combination with object-oriented

metrics such as CK metrics to improve the prediction ability of the independent variables.

The ISO-IEC 25023 standard [65] has recommended several metrics for product quality attributes identified in ISO-IEC 25010. However, one of the challenges of using the recommended metrics from ISO-IEC 25023 is that some are abstract and lack direct application to source code [66]. This may explain why none of the included secondary studies reported any of the recommended metrics from ISO-IEC 25023. The results have focused on popular metric suites such as CK, Halstead, Li & Henry metrics for reliability and maintainability. S02 suggests that other metrics not part of metric suites, such as LOCQ [67] have been noted to be effective in fault-prediction and may be explored in further studies.

6.6. Metrics validation

Since we have chosen to include the validity status mentioned by the secondary studies only, the validation status of source code metrics presents only the perspective available through the secondary studies. We acknowledge that using the validation status from other sources will reflect and change the results presented in Tables 11, 12, 16, and 17. Metric suites such as CK metrics [68], Li & Henry metrics [69], MOOD metrics [70] and QMOOD metrics [71] have been empirically validated [41,72–75]. However, all the good indicators metrics pointed out in the results (Table 13) remain the same as we have included both “high” and “moderate” strength of evidence. Incidentally, all source code metrics mentioned in Table 13 are validated based on the information available in literature.

Certain cohesion metrics, such as LCOM, have been highlighted as theoretically invalid metric for cohesion by Kitchenham et al. (S14) and thus may be considered less effective in capturing the cohesiveness of source code. Based on our findings, we suggest LCOM1 and LCOM2 are more suitable cohesion metrics and indicators of reliability.

6.7. Accuracy measures

The mean magnitude of residual error (MMRE) has been predominantly used in the secondary studies reporting explanatory results related to maintainability and reliability. The MMRE accuracy measure has been highlighted as biased in the literature [76,77]. Additionally, if primary studies use MMRE as a goodness of fit criteria for model evaluation, as well as to report the prediction accuracy, this bias may increase [78,79]. Another reason for MMRE being biased is that the measure does not define an upper bound for MMRE values greater than one, while MMRE values lower than one cannot become negative; thus, results of MMRE from models that underestimate seem more promising. Foss et al. [80] recommend using accuracy measures after considering the characteristics of the data. Since we can only report the accuracy measures provided in the secondary studies, we acknowledge that data related to data-distribution is not reported in our tertiary study.

6.8. Prediction models

In S01, Malhotra et al. note that machine learning models outperform statistical models and recommend exploring hybrid prediction models for maintainability prediction. Hybrid prediction models and ensemble learners are also recommended in S07 as well. Elmidaoui et al. (S04) further note that the accuracy of prediction models has been reported to be highly dependent on datasets used in addition to the independent variables used. Two secondary studies have investigated changing the datasets while using the same source code metrics as independent variables and modelling techniques report dependency on the datasets used. Prediction models that are accurate for a variety of datasets may be considered more robust. According to S05, the selection of source code metrics as independent variables for prediction models can be based on feature selection methods to utilize only those source code metrics which contribute to the improvement of the model and can be considered to be effective predictors.

6.9. Datasets used

In our included secondary studies, the datasets used in explanatory studies have been predominantly UIMS, and QUES datasets (e.g., S01 and S05) among other datasets. S05 point out that validation of prediction models on industrial datasets may further improve the validity of the prediction models and may be more relevant for practitioners. Large datasets based on languages used in practice may also increase the relevance of the maintainability prediction research. In this regard, S04 highlights the need for curated datasets (such as PROMISE dataset). Additionally, Radjenović et al. (S12) suggest that realistic datasets are often unbalanced, with faults occurring randomly among the software modules. Similarly, major restructuring between two software versions may change the distribution of faults. To produce accurate fault-prediction models, the availability of realistic unbalanced datasets is needed for model training and validation.

6.10. Consistent tool support

Measurement tools used across secondary studies are not the same. Thus, measurement error is possible when different tools are used to evaluate the same datasets across studies. In S06, Tiwari et al. [81] point out that consistent and dependable tools for coupling and cohesion metrics are needed for reliable evaluation of source code metrics.

6.11. Conducting tertiary review challenges

We also want to report challenges faced during data extraction phase which led to exclusion of relevant studies. While some secondary studies report the evidence between external attributes and source code metrics, the method of aggregation utilized in the secondary study makes it difficult to trace how many primary studies have reported the relationship between specific code metrics and external measures. Hosseini et al. [82] provide a synthesis of cross project defect prediction studies. The meta-analysis performed is seminal work in our opinion. However, the accuracy measures reported for different metrics could not be traced to individual metrics along with primary studies reporting the results. Similarly, Arvanitou et al. [8] suggest certain metrics to be effective in measuring maintainability, stability and changeability though do not provide number of primary studies reporting the trend for individual metrics. Since, one of the criterion used to establish strength of evidence was primary studies reporting individual metrics, these secondary studies were excluded from our analysis.

Another challenge faced during metrics aggregation is the non-standard method used when naming source code metrics. The inconsistency in naming of code metrics leads to several issues where the same metric is described by two acronyms or when same acronyms are assigned to two different metrics. This observation is shared by other researchers as well including Malhotra et al. (S07), Sharkawy et al. (S03), and Saraiva et al. [83]. To ameliorate the inconsistency, we also suggest an online catalog of source code metrics can be created which consistently reports metric descriptions, validation levels, along with alternate acronyms, and related internal attributes. This can be a difficult task since several secondary studies such as [21] have reported 300 metrics related to different programming paradigms and [83] report 575 object-oriented metrics for maintainability alone. Though, it could be a very effective tool for researchers, software tool developers and practitioners in the field.

While performing the quality assessment for the secondary studies, we identified several instances that are not explicitly covered by the DARE quality assessment criteria. To assess the search coverage sufficiency, the criteria assigns “yes” to secondary studies that have “searched four or more digital libraries and included additional search strategies OR identified and referenced all journals addressing the topic of interest” and “partly” to “searched three or four digital libraries with no extra strategies OR searched a defined but restricted set of journals

and conference proceedings”. The cases where a secondary study has searched in three or four digital libraries with extra search strategies (e.g., contacting the authors, snowballing) or when secondary studies have searched in five databases without any extra search strategy is open to interpretation and subject to the researcher conducting the assessment. Similarly, during assessment of quality of the secondary study, we noticed that some authors have explicitly defined the quality criteria and reported the results whereas others only defined the criteria and not reported any results, such as, S07, possibly due to page limitations. Since it is often the case that more than one author is involved in the quality assessment, such instances where subject assessment needs to be used, may lead to confusion and disagreements between the authors. We suggest that such issues maybe updated in the DARE criteria and software engineering community may evaluate other quality assessment tools derived from mature fields such as medicine, e.g., [49], to improve the quality assessment of secondary and tertiary studies. We also suggest that tertiary studies should not solely rely on DARE for assessing the quality of included secondary studies. DARE does not consider the strength of evidence of the results reported in secondary studies, since it does not critically appraise the quality assessment performed by the secondary studies. To perform an in-depth evaluation of evidence from secondary studies, future tertiary studies could use a similar method to our strength of evidence criteria (see Section 3.5.4), to complement DARE.

6.12. Future works

Other researchers can utilize different criteria for the strength of evidence on the data available online.³ They can customize the results to specific usage scenarios. Additionally, researchers can extend the same schema of evaluating the strength of evidence to other secondary studies that have not been considered in our tertiary study.

We have also highlighted that no secondary studies reported links between source code metrics and external quality attributes in the aspect-oriented programming paradigm which is a research gap that needs to be investigated. The link between dynamic metrics and external attributes has not been reported in the secondary studies considered. Investigating the link between dynamic metrics and external attributes can be an interesting theme for future research. Similarly, the use of process metrics in addition to object-oriented metrics may be explored for suitability in assessing quality attributes in highly iterative development environments.

We further suggest that a catalog of source code metrics with consistent terminology and description can benefit practitioners, tool developers, and researchers. Researchers can utilize the list of secondary studies that only report using source code metrics and external quality attributes as a starting point for creating a catalog of source code metrics. Keeping in view the limitations raised in Section 7, we suggest further work to bridge the gap between syntactic structural information and semantic assessment of internal quality attributes. Fregnan et al. [84] have summarized semantic coupling metrics using machine learning models to link similar words and documents. This may help develop semantic cohesion metrics.

7. Limitations

We follow suggestions by Kitchenham et al. [48] and discuss the intentional deviations from the systematic review guidelines [31,33] followed in the design of the tertiary study. We also discuss possible implications of these deviations on the study outcomes. In addition, we discuss the threat of double counting due to overlapping primary studies (see Section 7.3).

7.1. Study selection

We have aimed to cover a large corpus of knowledge, however, there is a possibility that we have missed a small percentage of relevant secondary studies that were not in English language, or due to limitations of keywords in our search string and no snowballing on the final set of secondary studies. We believe that such number of secondary studies are less compared to the included set of secondary studies and unlikely to significantly alter the results presented.

7.2. Research validity

We acknowledge the limitation of using the DARE quality score as a measure contributing to the strength of evidence. These scores represent the quality of a secondary study and will abstract away the individual primary studies’ quality. A deeper analysis can be performed where we utilize the quality of the primary studies reporting the link between code metrics and quality attributes.

In Table 13, we have suggested source code metrics that are good indicators of external quality attributes. Several previous studies have highlighted that source code metrics measuring the same internal quality attribute can be correlated [85]. Source code metrics that measure different internal quality attributes have also been reported as correlated [86,87].

Source code metrics capture syntactical structural information and thus do not capture the source code’s semantical complexity, which limits their applicability in assessing program comprehension.

Similarly, we note that the semantic cohesion of source code may differ from syntactic structure information related to cohesion and has not been discussed in the included secondary studies. While developers in practice may perceive semantic cohesion differently in given contexts, they use the concept of logical cohesion by preferring to place classes and functions with similar functionality in the same software packages [9]. The relationship between syntactic cohesion and external quality attributes needs to be characterized for specific contexts. As an example, while designing libraries with particular functionality may be considered good design practice, accessing attributes or methods from a library may lower the overall cohesion of the calling class, thus giving a wrongful impression of cohesion and external quality attribute being measured. Therefore, we suggest utilizing the evidence presented in our study in light of such limitations of source code metrics.

When using source code metrics for coupling, the coupling direction needs to be considered for a meaningful utilization of the coupling metrics. Arisholm et al. [88] discussed how “control classes”, i.e., large classes with the bulk of functionality that depend on several smaller classes for ancillary functionality, may be easier to modify for inexperienced developers. Kitchenham et al. [89] suggested CBO to be invalid as it treats inward and outward coupling similarly.

In practice, large classes may become a “bottleneck” when they have both high outward coupling and high inward coupling. The coupling metrics reported in Table 13 do not capture such nuances. Practitioners should prioritize classes with high outward and high inward coupling for refactoring or redesign, as small changes in them may lead to multiple changes in classes linked to them.

7.3. Double counting

When conducting tertiary studies, there is a risk of double counting when the same primary studies are included in more than one secondary study [43]. To address the threat of double counting, we identified the secondary studies with overlap of primary studies. For such cases, we ensured that the duplicate primary studies were considered only once so that the results are not inflated due to double counting.

Table 16

Strength of evidence: reliability linked with code metrics in exploratory studies.

External metric	Internal attribute	Metrics	Studies	Overlap in PS	DARE score	Relationship indicator							Strength of evidence
						“++”	“+”	“0”	“-”	“--”	Unclear	Sig*	
Fault-proneness	Coupling	CBO	S06V, S12	7 PS, Resolved	M, H	48	–	–	–	–	–	4	High
	Inheritance	DIT	S06V, S12	7 PS, Resolved	M, H	0	–	52	–	–	–	3	High
	Cohesion	LCOM	S06V, S12	7 PS, Resolved	M, H	0	–	66	–	–	–	4	High
	Size	LOC	S12, S14V	1 PS, Resolved	H, M	–	61	–	–	–	–	–	High
	Inheritance	NOC	S06V, S12	7 PS, Resolved	M, H	–	–	53	–	–	–	3	High
	Complexity	RFC	S06V, S12	7 PS, Resolved	M, H	51	–	–	–	–	–	5	High
	Complexity	WMC	S06V, S12	7 PS, Resolved	M, H	44	–	–	–	–	–	4	High
	Complexity	Cyclomatic Complexity	S12	–	H	–	43	–	–	–	–	–	High
Fault-proneness	Code Churn	Age	S12	–	H	10	–	–	–	–	–	–	Moderate
	Coupling	CBO	S13V	–	H	–	–	1	–	–	–	23	Moderate
	Code Churn	Changes	S12	–	H	16	–	–	–	–	–	–	Moderate
	Code Churn	Churn	S12	–	H	15	–	–	–	–	–	–	Moderate
	Inheritance	DIT	S13V	–	H	–	–	15	–	–	–	9	Moderate
	Size	Halstead N1	S12	–	H	–	–	12	–	–	–	–	Moderate
	Size	Halstead n1	S12	–	H	–	–	12	–	–	–	–	Moderate
	Size	Halstead N2	S12	–	H	–	–	12	–	–	–	–	Moderate
	Size	Halstead n2	S12	–	H	–	–	12	–	–	–	–	Moderate
	Cohesion	LCOM	S13V	–	H	–	–	4	–	–	–	15	Moderate
	Inheritance	NOC	S13V	–	H	–	–	15	–	–	–	3	Moderate
	Maturity	Past Faults	S12	–	H	–	10	–	–	–	–	–	Moderate
	Complexity	RFC	S13V	–	H	–	–	–	–	–	–	24	Moderate
	Size	SLOC	S13V	–	H	–	–	–	–	–	–	16	Moderate
	Complexity	WMC	S13V	–	H	–	–	1	–	–	–	21	Moderate
Not Reported in SS	Complexity	AMC	S09	–	H	–	–	3	–	–	–	35	Moderate
	Coupling	CBO	S09	–	H	–	–	3	–	–	20	60	Moderate
	Inheritance	DIT	S09	–	H	–	–	34	–	–	25	20	Moderate
	Cohesion	LCOM1	S09	–	H	–	–	17	–	–	21	33	Moderate
	Cohesion	LCOM2	S09	–	H	–	–	10	–	–	9	21	Moderate
	Cohesion	LCOM3	S09	–	H	–	–	8	2	–	–	10	Moderate
	Cohesion	LCOM4	S09	–	H	–	–	9	1	–	2	5	Moderate
	Cohesion	LCOM5	S09	–	H	–	–	2	3	–	2	12	Moderate
	Size	LOC	S09	–	H	–	–	0	1	–	–	58	Moderate
	Inheritance	NOC	S09	–	H	–	–	31	1	–	25	22	Moderate
	Complexity	NOM	S09	–	H	–	–	5	–	–	1	16	Moderate
	Size	NPM	S09	–	H	–	–	5	–	–	2	6	Moderate
	Coupling	OCAEC	S09	–	H	–	–	6	1	–	–	3	Moderate
	Coupling	OCAIC	S09	–	H	–	–	1	–	–	–	10	Moderate
	Coupling	OCMEC	S09	–	H	–	–	7	–	–	–	4	Moderate
	Coupling	OCMIC	S09	–	H	–	–	6	–	–	–	5	Moderate
	Coupling	OMMEC	S09	–	H	–	–	6	–	–	–	4	Moderate
	Coupling	OMMIC	S09	–	H	–	–	–	–	–	–	10	Moderate
	Complexity	RFC	S09	–	H	–	–	5	–	–	21	50	Moderate
	Cohesion	TCC	S09	–	H	–	–	11	7	–	2	1	Moderate
	Complexity	VG (McCabe)	S09	–	H	–	–	–	–	–	1	32	Moderate
	Complexity	WMC-McCabe	S09	–	H	–	–	1	1	–	19	62	Moderate
Defects	Complexity	Cyclomatic Complexity	S10	–	H	–	1	–	–	–	–	–	Low
Fault-proneness	Code Churn	Change Set	S12	–	H	8	–	–	–	–	–	–	Low
Not Reported in SS	Coupling	MPC	S09	–	H	–	8	0	–	–	1	–	Low

Sig*: Study S13 does not distinguish between positive or negative significance when reporting significantly linked source code metrics.

8. Conclusions

In this tertiary study, we have performed a systematic review of secondary studies that have evidence for the link between source code metrics and external quality attributes as classified by prevailing software quality standards. After an extensive search, we identified 15 exploratory and explanatory secondary studies of moderate and high quality on the subject. The linked evidence considered includes qualitative and quantitative results of the link between source code metrics and external quality attributes. We excluded secondary studies that did not report any stated evidence of a relationship between source code metrics and external quality attributes.

Results from moderate and high-quality secondary studies show that evidence of a link is only available for reliability, maintainability, and security. In contrast, other external quality attributes have not been linked with source code metrics in the included secondary studies. Only one secondary study reported a link with a security sub-attribute,

highlighting the need for further studies exploring how source code metrics are linked with security and other external quality attributes.

The evidence shows that source code metrics have a varied link with external attributes depending on the external metric used as the dependent variable for external quality attributes. After evaluating individual results from the perspective of the overall strength of evidence, using the quality of secondary studies and the number of primary studies reporting the results, we report consistent results for a limited set of source code metrics and internal quality attributes. Good indicators of external attributes are provided in Table 13. Several source code metrics have been observed to have an insignificant or unclear link with both maintainability and reliability.

Our results aid in developing confidence in the metrics reported as good indicators, which may be useful for future studies on the subject. Source code metrics categorized as consistent good indicators of external quality attributes can aid practitioners in focusing on specific source code metrics to assess the external quality attributes of interest. Source

Table 17
Strength of evidence: reliability linked with code metrics in explanatory studies.

External metric	Metrics sets	Prediction model	Studies	DARE Score	Relationship indicator		Strength of evidence
					“+”	Unclear	
Fault-prediction	6 Halstead, McCabe, LOC and Branch	CCA, NB, BAG, LR and BO	S11	M	1		Low
	BRS, EBS, EBC, BP, BS and BR	RF, SVM and MP	S11	M	1		Low
	CBO, DEPTH, LCOM and WMC	BN.	S11	M	1		Low
	CK metrics	NB and J48	S11	M		1	Low
	CK & QMOOD metrics	LR, UR, MR, NN, DT, SVM, BST, RF, BAG and MP.	S11	M	1		Low
	CK metrics	J48, NB.	S11	M	1		Low
	Halstead, McCabe, LOC, CK-suite & Misc	MOPSO-N, NN, BN, NB, SVM and DT(C45).	S11	M	1		Low
	Halstead, McCabe & LOC	AFP and ALS	S11	M	1		Low
		AR, DT, k-NN, NBC, SVM, BAG and BST	S11	M		1	Low
		CC, NN-filter and TNB.	S11	M	1		Low
		DT and K-means.	S11	M	1		Low
		DT(C4.5), RF, AdaCost, Adc2, Csb2, MetaCost, Weighting, and RUS.	S11	M	1		Low
		J48 Tree, DWT and PCA.	S11	M	1		Low
		K-Means and X Means.	S11	M	1		Low
		KNN and SVM.	S11	M	1		Low
		LR, PR, SVR, NN, SVLR, NB and J48 Tree.	S11	M	1		Low
		MLR.	S11	M	1		Low
		NB and DT(C4.5)	S11	M	1		Low
		NB and DT(J48).	S11	M	1		Low
		NB, J48 and OneR	S11	M	1		Low
		NB, SVM and NN.	S11	M	1		Low
		RANQ, NB, MLP, K-NN and LR	S11	M	1		Low
		RF	S11	M	1		Low
		RF, BAG, LR, BST and NB	S11	M	1		Low
		RF, BAG, LR, BST and NB.	S11	M	1		Low
		RF.	S11	M	1		Low
		SA, ACP, SVM and ANN.	S11	M	1		Low
ClusteringSK-means, Clustering, MDBC, RF, NB and J48 Tree.		S11	M		1	Low	
NB and RF		S11	M		1	Low	
NB, DT(C4.5-J48), SVM and LR.		S11	M		1	Low	
RF, BAG, LR, BST, NB, Jrip, Ibk, J48, Decorate and AODE		S11	M		1	Low	
Halstead, McCabe, LOC and Branch	RF, LDF and LR.	S11	M	1		Low	
Halstead, McCabe, LOC & Misc	ANN, SVM and DT and CCN.	S11	M	1		Low	
LOC, CBO, LOCQ, WMC, RFC, LCOM, LCOM3, DIT & NOC	CA, LR, J48 and NB.	S11	M		1	Low	
OO & McCabe	BN	S11	M	1		Low	
	RF, LR, NB and DT.	S11	M		1	Low	

Table 18
List of selected secondary studies.

#	Title	Study type	Venue	Year	Ref
S01	A systematic literature review on empirical studies towards prediction of software maintainability	SLR	Soft Computing	2020	Malhotra and Lata [90]
S02	Machine Learning Techniques for Software Bug Prediction: A Systematic Review	SLR	Journal of Computer Science	2020	Saharudin et al. [91]
S03	Metrics for analysing variability and its implementation in software product lines: A systematic literature review	SLR	Information & Software Technology	2019	El-Sharkawy et al. [92]
S04	Empirical studies on software product maintainability prediction: A systematic mapping and review	SMS	E-Informatica	2019	Elmidaoui et al. [93]
S05	Software change prediction: A systematic review and future guidelines	SLR	E-Informatica	2019	Malhotra and Khanna [94]
S06	Coupling and cohesion metrics for object-oriented software: A systematic mapping study	SMS	Innovations in Software Engineering Conference	2018	Tiwari and Rathore [81]
S07	Software Maintainability: Systematic Literature Review and Current Trends	SLR	International Journal of Software Engineering & Knowledge Engineering	2016	Malhotra and Chug [95]
S08	Software change prediction: A literature review	SLR	International Journal of Computer Applications in Technology	2016	Malhotra and Bansal [96]
S09	Empirical evidence on the link between object-oriented measures and external quality attributes: A systematic literature review	SLR	Empirical Software Engineering	2015	Jabangwe et al. [52]
S10	How have we evaluated software pattern application? A systematic mapping study of research design practices	SMS	Information & Software Technology	2015	Riaz et al. [97]

(continued on next page)

Table 18 (continued).

#	Title	Study type	Venue	Year	Ref
S11	Software fault prediction: A systematic mapping study	SMS	Ibero-American Conference on Software Engineering	2016	Murillo-Morera et al. [98]
S12	Software fault prediction metrics: A systematic literature review	SLR	Information & Software Technology	2013	Radjenović et al. [99]
S13	A systematic review of the empirical validation of object-oriented metrics towards fault-proneness prediction	SLR	International Journal of Software Engineering & Knowledge Engineering	2013	Isong and Obeten [100]
S14	What is up with software metrics? — A preliminary mapping study	SMS	Journal of Systems & Software	2010	Kitchenham [101]
S15	A systematic review of software maintainability prediction and metrics	SLR	International Symposium on Empirical Software Engineering & Measurement	2009	Riaz et al. [15]

Table 19

Interpretation of the DARE criteria by Budgen et al. [45].

Criterion	Score	Interpretation
Inclusion and exclusion	yes	The criteria used are explicitly defined in the paper
	partly no	The inclusion/exclusion criteria are implicit The criteria are not defined and cannot be readily inferred.
Search coverage	yes	The authors have searched four or more digital libraries and included additional search strategies OR identified and referenced all journals addressing the topic of interest.
	partly	Searched three or four digital libraries with no extra search strategies OR searched a defined but restricted set of journals and conference proceedings.
	no	Searched up to two digital libraries or an extremely restricted set of journals.
Assessment of quality	yes	The authors have explicitly defined quality criteria and extracted them from each primary study
	partly	The research question involved quality issues that are addressed by the study
	no	No explicit quality assessment of individual papers has been attempted
Study description	yes	Detailed information is presented about each study
	partly no	Only summary information is presented about the studies Details of the studies are not provided
Synthesis of studies	yes	The authors have performed a meta-analysis or used another form of synthesis for all the data of the study
	partly	Synthesis has been performed for some of the data from some of the primary studies
	no	No explicit synthesis has been performed (as in a mapping study)

code metrics that are insignificantly linked may be investigated for a link with other external attributes of quality.

In our analysed secondary studies, the data sets used for validation impact the effectiveness of code metrics-based prediction models. However, GMDH-based prediction models have performed better than other models considered on several data sets.

Software development is essentially a human-centric activity. An interesting future research direction can be evaluating the link between source code metrics and external quality attributes under different development processes, e.g., continuous integration where source code undergoes regular updates due to a deployment-oriented outlook. We also note that evidence of the link has been context-sensitive. Future studies can aim to solidify evidence of the link for specific development contexts, e.g., web development, databases development, or mobile development. Thus, we can compare structurally and semantically similar source code and use the source code metrics to assess product quality objectively.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data is shared on zenodo .org with a permanent link attached as a footnote in the article.

Acknowledgement

This work has been supported by ELLIIT, a Strategic Area within IT and Mobile Communications, funded by the Swedish Government. The work has also been supported by research grant for the OSIR project (reference number 20190081) from the Knowledge Foundation in Sweden.

Table 20
External quality attributes and external metrics used in the secondary studies.

Study ID	External quality attribute	External metric	Description
S01	Maintainability	Change	Lines of code added, deleted, and modified
S02	Reliability	Bugs	Not provided in the secondary study
S03	Maintainability	Change impact	Not provided in the secondary study
S03	Vulnerability	Not Reported	Number of internal #ifdef-blocks, Number of internal configurations options, Number of external configuration options, degree centrality, eigenvector centrality, betweenness centrality
S04	Maintainability	Change	The number of changes made to the source code (changes in LOC, change in modules, change in class)
S05	Maintainability	Change-proneness	Change-proneness is defined as the likelihood that a class would change across different versions of a software product
S06	Reliability	Fault proneness	The probability of detecting faulty classes
S06	Maintainability	Maintenance Effort	The maintenance effort is measured by the number of lines changed per class
S07	Maintainability	Change	Not defined by secondary study. Only referred to as 'change' in the inclusion/exclusion criteria
S08	Maintainability	Change	Not defined in the secondary study. Only referred to as "change prediction" in the introduction
S09	Maintainability & Reliability	Multiple external metrics	The secondary study combines evidence for link between external quality attribute and source code metrics from primary studies that use multiple external metrics to measure the external quality attribute. Also, the definition of the external metric are not provided.
S10	Reliability	Defects	Not provided in the secondary study
S11	Reliability	Fault-prediction	Estimating the number of defects remaining in software systems
S12	Reliability	Fault proneness	Fault prediction models are used to improve software quality and to assist software inspection by locating possible faults. A correct service is delivered when the service implements the system function. A service failure is an event that occurs when the delivered service deviates from the correct/expected service. The deviation is called an error. The adjudged or hypothesized cause of an error is called a fault
S13	Reliability	Fault proneness	The paper does not define fault proneness, however, mentions "Defects during development are inevitable and the earlier they are found and fixed, the lesser it costs and the higher the quality of the products delivered"
S14	Reliability	Fault proneness	The probability of exhibiting a fault
S15	Maintainability	Maintainability Index	Not defined in the secondary study. In the primary studies of S15 it is defined as $MI = 171 - 5.2 \ln(\text{average Halstead volume}) - 0.23 (\text{average extended cyclomatic complexity per module}) - 16.2 \ln(\text{average count of lines of source code per module}) + 50 \sin(\sqrt{2.4 * (\text{average percentage of lines of comments per module})})$

Appendix

See Tables 16–20.

References

- [1] B. Kitchenham, S. Pfleeger, Software quality: the elusive target [special issues section], *IEEE Softw.* 13 (1) (1996) 12–21, <http://dx.doi.org/10.1109/52.476281>, Conference Name: IEEE Software.
- [2] IEEE, 1074-2006—IEEE Standard for Developing a Software Project Life Cycle Process, IEEE, 2006.
- [3] J.A. McCall, P.K. Richards, G.F. Walters, Factors in Software Quality, Volumes I, II, and III, US Rome Air Development Center Reports, US Department of Commerce, USA, 1977.
- [4] B.W. Boehm, J.R. Brown, M. Lipow, Quantitative evaluation of software quality, in: Proceedings of the 2nd International Conference on Software Engineering, 1976, pp. 592–605.
- [5] R.G. Dromey, A model for software product quality, *IEEE Trans. Softw. Eng.* 21 (2) (1995) 146–162.
- [6] Organización Internacional de Normalización, ISO-IEC 25010: 2011 Systems and Software Engineering-Systems and Software Quality Requirements and Evaluation (SQuARE)-System and Software Quality Models, International Organization for Standardization, ISO, Geneva, Switzerland, 2011.
- [7] S. Montagud, S. Abrahão, E. Insfran, A systematic review of quality attributes and measures for software product lines, *Softw. Qual. J.* 20 (3–4) (2012) 425–486, <http://dx.doi.org/10.1007/s11219-011-9146-7>.
- [8] E. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou, M. Galster, P. Avgeriou, A mapping study on design-time quality attributes and metrics, *J. Syst. Softw.* 127 (2017) 52–77, <http://dx.doi.org/10.1016/j.jss.2017.01.026>.
- [9] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, CRC Press, 2019.
- [10] J.P. Miguel, D. Mauricio, G. Rodriguez, A review of software quality models for the evaluation of software products, *Int. J. Softw. Eng. Appl.* 5 (6) (2014) 31–53, <http://dx.doi.org/10.5121/ijsea.2014.5603>, arXiv:1412.2977.
- [11] H.A. Al-Jamimi, M. Ahmed, Prediction of software maintainability using fuzzy logic, in: 2012 IEEE International Conference on Computer Science and Automation Engineering, IEEE, 2012, pp. 702–705.
- [12] S. Almugrin, W. Albattah, O. Alaql, M. Alzahrani, A. Melton, Instability and abstractness metrics based on responsibility, in: 2014 IEEE 38th Annual Computer Software and Applications Conference, IEEE, 2014, pp. 364–373.
- [13] G. Concas, M. Marchesi, A. Murgia, S. Pinna, R. Tonelli, Assessing traditional and new metrics for object-oriented systems, in: Proceedings of the 2010 ICSE Workshop on Emerging Trends in Software Metrics, 2010, pp. 24–31.
- [14] L.C. Briand, J. Wüst, Empirical studies of quality models in object-oriented systems, *Adv. Comput.* 56 (2002) 97–166.
- [15] M. Riaz, E. Mendes, E. Tempero, A systematic review of software maintainability prediction and metrics, in: 2009 3rd International Symposium on Empirical Software Engineering and Measurement, IEEE, Lake Buena Vista, FL, USA, 2009, pp. 367–377, <http://dx.doi.org/10.1109/ESEM.2009.5314233>.
- [16] M.T. Baldassarre, D. Caivano, S. Romano, G. Scanniello, Software models for source code maintainability: A systematic literature review, in: 2019 45th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, Kallithea-Chalkidiki, Greece, 2019, pp. 252–259, <http://dx.doi.org/10.1109/SEAA.2019.00047>.

- [17] J. Saraiva, S. Soares, F. Castor, Towards a catalog of object-oriented software maintainability metrics, in: 2013 4th International Workshop on Emerging Trends in Software Metrics, WETSoM, IEEE, San Francisco, CA, 2013, pp. 84–87, <http://dx.doi.org/10.1109/WETSoM.2013.6619342>.
- [18] R. Abilio, P. Teles, H. Costa, E. Figueiredo, A systematic review of contemporary metrics for software maintainability, in: 2012 Sixth Brazilian Symposium on Software Components, Architectures and Reuse, IEEE, Natal, Brazil, 2012, pp. 130–139, <http://dx.doi.org/10.1109/SBCARS.2012.15>.
- [19] R. Malhotra, A. Chug, Software maintainability: Systematic literature review and current trends, *Int. J. Softw. Eng. Knowl. Eng.* 26 (08) (2016) 1221–1253, <http://dx.doi.org/10.1142/S0218194016500431>.
- [20] R. Burrows, A. Garcia, F. Taiani, Coupling metrics for aspect-oriented programming: A systematic review of maintainability studies, *Eval. Nov. Approaches Softw. Eng.* (2009) 277–290.
- [21] A.S. Nuñez-Varela, H.G. Pérez-Gonzalez, F.E. Martínez-Perez, C. Soubervielle-Montalvo, Source code metrics: A systematic mapping study, *J. Syst. Softw.* 128 (2017) 164–197, <http://dx.doi.org/10.1016/j.jss.2017.03.044>.
- [22] ISO, IEC 9126-1: Software Engineering-Product Quality-Part 1: Quality Model, Vol. 21, International Organization for Standardization, Geneva, Switzerland, 2001.
- [23] R.B. Grady, D.L. Caswell, *Software Metrics: Establishing a Company-Wide Program*, Prentice-Hall, Inc., 1987.
- [24] M. Soto, M. Ciolkowski, The qualoss open source assessment model measuring the performance of open source communities, in: 2009 3rd International Symposium on Empirical Software Engineering and Measurement, 2009, pp. 498–501, <http://dx.doi.org/10.1109/ESEM.2009.5314237>.
- [25] A. Mayr, R. Plösch, M. Kläs, C. Lampasona, F. Iese, M. Saft, A comprehensive code-based quality model for embedded systems, in: 2012 IEEE 23rd International Symposium on Software Reliability Engineering, 2012, p. 10.
- [26] K. Mordal-Manet, F. Balmas, S. Denier, S. Ducasse, H. Wertz, J. Laval, F. Bellengard, P. Vaillergues, The squal model — A practice-based industrial quality model, in: 2009 IEEE International Conference on Software Maintenance, IEEE, Edmonton, AB, Canada, 2009, pp. 531–534, <http://dx.doi.org/10.1109/ICSM.2009.5306381>.
- [27] H. Rashidi, M. Sadeghzadeh Hemayati, Software quality models: A comprehensive review and analysis, *J. Electr. Comput. Eng. Innov.* 6 (1) (2018) <http://dx.doi.org/10.22061/jeccei.2019.1076>.
- [28] CISQ Specifications for Automated Quality Characteristic Measures, CISQ Technical Work Groups for Reliability, Performance Efficiency, Security, Maintainability, Standard CISQ-TR 2012-01, OMG, 2012.
- [29] A. Romanovsky, T. Vardanega, Reliable Software Technologies–Ada-Europe 2011: 16th Ada-Europe International Conference on Reliable Software Technologies, Edinburgh, UK, June 20–24, 2011. Proceedings, Vol. 6652, Springer Science & Business Media, 2011.
- [30] G. Lacerda, F. Petrillo, M. Pimenta, Y.G. Guéhéneuc, Code smells and refactoring: a tertiary systematic review of challenges and observations, *J. Syst. Softw.* (2020) 110610.
- [31] B.A. Kitchenham, D. Budgen, P. Brereton, *Evidence-Based Software Engineering and Systematic Reviews*, Vol. 4, CRC Press, 2015.
- [32] N.B. Ali, M. Usman, Reliability of search in systematic reviews: Towards a quality assessment framework for the automated-search strategy, *Inf. Softw. Technol.* 99 (2018) 133–147.
- [33] K. Petersen, S. Vakkalanka, L. Kuzniarz, Guidelines for conducting systematic mapping studies in software engineering: An update, *Inf. Softw. Technol.* 64 (2015) 1–18.
- [34] L. Chen, M.A. Babar, H. Zhang, Towards an evidence-based understanding of electronic data sources, in: 14th International Conference on Evaluation and Assessment in Software Engineering, EASE, 2010, pp. 1–4.
- [35] M. Turner, *Digital Libraries and Search Engines for Software Engineering Research: an Overview*, Keele University, UK, 2010.
- [36] J.L. Barros-Justo, F.B. Benitti, S. Matalonga, Trends in software reuse research: A tertiary study, *Comput. Stand. Interfaces* 66 (2019) 103352.
- [37] N.b. Ali, B. Tanveer, A comparison of citation sources for reference and citation-based search in systematic literature reviews, *e-Inf. Softw. Eng. J.* (2022).
- [38] H.K.V. Tran, J. Börstler, N. bin Ali, M. Unterkalmsteiner, How good are my search strings? Reflections on using an existing review as a quasi-gold standard, *e-Inf. Softw. Eng. J.* 16 (1) (2022).
- [39] N.B. Ali, K. Petersen, Evaluating strategies for study selection in systematic literature studies, in: Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, 2014, pp. 1–4.
- [40] J.R. Landis, G.G. Koch, The measurement of observer agreement for categorical data, *Biometrics* (1977) 159–174.
- [41] K. El Emam, Benchmarking Kappa: Interrater agreement in software process assessments, *Empir. Softw. Eng.* 4 (2) (1999) 113–133.
- [42] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in software engineering, in: 12th International Conference on Evaluation and Assessment in Software Engineering, EASE 12, 2008, pp. 1–10.
- [43] J. Börstler, N.B. Ali, K. Petersen, Double-counting in software engineering tertiary studies - An overlooked threat to validity, *Inf. Softw. Technol.* 158 (2023) 107174, <http://dx.doi.org/10.1016/j.infsof.2023.107174>.
- [44] N.B. Ali, M. Usman, A critical appraisal tool for systematic literature reviews in software engineering, *Inf. Softw. Technol.* 112 (2019) 48–50.
- [45] D. Budgen, P. Brereton, N. Williams, S. Drummond, What support do systematic reviews provide for evidence-informed teaching about software engineering practice? *e-Inf. Softw. Eng. J.* 14 (1) (2020) 7–60.
- [46] R. Hoda, N. Salleh, J. Grundy, H.M. Tee, Systematic literature reviews in agile software development: A tertiary study, *Inf. Softw. Technol.* 85 (2017) 60–70.
- [47] K. Curcio, R. Santana, S. Reinehr, A. Malucelli, Usability in agile software development: A tertiary study, *Comput. Stand. Interfaces* 64 (2019) 61–77.
- [48] B. Kitchenham, L. Madeyski, D. Budgen, SEGRESS: Software engineering guidelines for reporting secondary studies, *IEEE Trans. Softw. Eng.* 49 (3) (2022) 1273–1298.
- [49] M. Usman, N.B. Ali, C. Wohlin, A quality assessment instrument for systematic literature reviews in software engineering, *e-Inf. Softw. Eng. J.* 17 (1) (2023) 230105, <http://dx.doi.org/10.37190/e-inf230105>.
- [50] A.R. Gray, S.G. MacDonell, A comparison of techniques for developing predictive models of software metrics, *Inf. Softw. Technol.* 39 (6) (1997) 425–437.
- [51] S.D. Conte, H.E. Dunsmore, V.Y. Shen, *Software Engineering Metrics and Models*, Benjamin-Cummings Publishing Co., Inc., 1986.
- [52] R. Jabangwe, J. Börstler, D. Šmite, C. Wohlin, Empirical evidence on the link between object-oriented measures and external quality attributes: A systematic literature review, *Empir. Softw. Eng.* 20 (3) (2015) 640–693.
- [53] K. Sreeji, C. Lakshmi, A systematic literature review: Recent trends and open issues in software refactoring, *Int. J. Appl. Eng. Res.* 10 (18) (2015) 39696–39707.
- [54] M. Yan, X. Xia, X. Zhang, L. Xu, D. Yang, S. Li, Software quality assessment model: a systematic mapping study, *Sci. China Inf. Sci.* 62 (9) (2019) <http://dx.doi.org/10.1007/s11432-018-9608-3>.
- [55] M. Yan, X. Xia, X. Zhang, L. Xu, D. Yang, A systematic mapping study of quality assessment models for software products, in: 2017 International Conference on Software Analysis, Testing and Evolution, SATE, IEEE, 2017, pp. 63–71.
- [56] F.N. Colakoglu, A. Yazici, A. Mishra, Software product quality metrics: A systematic mapping study, *IEEE Access* 9 (2021) 44647–44670, <http://dx.doi.org/10.1109/ACCESS.2021.3054730>, Conference Name: IEEE Access.
- [57] Y. Khan, M. Elish, M. El-Attar, A Systematic Review on the Impact of CK Metrics on the Functional Correctness of Object-Oriented Classes, in: Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), vol. 7336 LNCS (no. PART 4) (2012) 258–273, http://dx.doi.org/10.1007/978-3-642-31128-4_19.
- [58] P. Morrison, D. Moye, R. Pandita, L. Williams, Mapping the field of software life cycle security metrics, *Inf. Softw. Technol.* 102 (2018) 146–159, <http://dx.doi.org/10.1016/j.infsof.2018.05.011>.
- [59] P. Saxena, M. Saini, Empirical studies to predict fault proneness: A review, *Int. J. Comput. Appl.* 22 (8) (2011) 41–45.
- [60] C. Catal, B. Diri, A systematic review of software fault prediction studies, *Expert Syst. Appl.* 36 (4) (2009) 7346–7354.
- [61] P. Oman, J. Hagemester, Metrics for assessing a software system's maintainability, in: Proceedings Conference on Software Maintenance 1992, IEEE Computer Society, 1992, pp. 337–338.
- [62] A. Tahir, S. MacDonell, A systematic mapping study on dynamic metrics and software quality, in: IEEE International Conference on Software Maintenance, ICSM, 2012, pp. 326–335, <http://dx.doi.org/10.1109/ICSM.2012.6405289>.
- [63] N. Nagappan, L. Williams, J. Osborne, M. Vouk, P. Abrahamsson, Providing test quality feedback using static source code and automatic test suite metrics, in: 16th IEEE International Symposium on Software Reliability Engineering, ISSRE'05, IEEE, 2005, pp. 85–94.
- [64] N. Nagappan, L. Williams, M. Vouk, J. Osborne, Using in-process testing metrics to estimate post-release field quality, in: The 18th IEEE International Symposium on Software Reliability, ISSRE '07, IEEE, 2007, pp. 209–214.
- [65] Organización Internacional de Normalización, ISO-IEC 25023: 2016-Systems and Software Engineering-Systems and software Quality Requirements and Evaluation (SQuARE)-Measurement of System and Software Product Quality, International Organization for Standardization, ISO, Geneva, Switzerland, 2016.
- [66] J. Spray, R. Sinha, A. Sen, X. Cheng, Building maintainable software using abstraction layering, *IEEE Trans. Softw. Eng.* 48 (11) (2021) 4397–4410.
- [67] A. Okutan, O.T. Yildiz, Software defect prediction using Bayesian networks, *Empir. Softw. Eng.* 19 (1) (2014) 154–181.
- [68] S.R. Chidamber, C.F. Kemerer, A metrics suite for object oriented design, *IEEE Trans. Softw. Eng.* 20 (6) (1994) 476–493.
- [69] M. Lorenz, J. Kidd, *Object-Oriented Software Metrics: A Practical Guide*, Prentice-Hall, Inc., 1994.
- [70] F.B. Abreu, R. Carapuça, Object-oriented software engineering: Measuring and controlling the development process, in: Proceedings of the 4th International Conference on Software Quality, Vol. 186, 1994, pp. 1–8.
- [71] J. Bansiya, C.G. Davis, A hierarchical model for object-oriented design quality assessment, *IEEE Trans. Softw. Eng.* 28 (1) (2002) 4–17.
- [72] L.C. Briand, S. Morasca, V.R. Basili, Property-based software engineering measurement, *IEEE Trans. Softw. Eng.* 22 (1) (1996) 68–86.

- [73] R. Harrison, S. Counsell, R. Nithi, An overview of object-oriented design metrics, in: *Proceedings Eighth IEEE International Workshop on Software Technology and Engineering Practice Incorporating Computer Aided Software Engineering*, IEEE, 1997, pp. 230–235.
- [74] M.O. Elish, A.H. Al-Yafei, M. Al-Mulhem, Empirical comparison of three metrics suites for fault prediction in packages of object-oriented systems: A case study of Eclipse, *Adv. Eng. Softw.* 42 (10) (2011) 852–859.
- [75] T. Mayer, T. Hall, Measuring OO systems: a critical analysis of the MOOD metrics, in: *Proceedings Technology of Object-Oriented Languages and Systems. TOOLS 29 (Cat. No. PR00275)*, IEEE, 1999, pp. 108–117.
- [76] M. Jørgensen, A critique of how we measure and interpret the accuracy of software development effort estimation, in: *First International Workshop on Software Productivity Analysis and Cost Estimation*, Citeseer, 2007, pp. 1–6.
- [77] B.A. Kitchenham, L.M. Pickard, S.G. MacDonell, M.J. Shepperd, What accuracy statistics really measure [software estimation], *IEE Proc.-Softw.* 148 (3) (2001) 81–85.
- [78] I. Myrtveit, E. Stensrud, M. Shepperd, Reliability and validity in comparative studies of software prediction models, *IEEE Trans. Softw. Eng.* 31 (5) (2005) 380–391.
- [79] I. Myrtveit, E. Stensrud, Validity and reliability of evaluation procedures in comparative studies of effort prediction models, *Empir. Softw. Eng.* 17 (2012) 23–33.
- [80] T. Foss, E. Stensrud, B. Kitchenham, I. Myrtveit, A simulation study of the model evaluation criterion MMRE, *IEEE Trans. Softw. Eng.* 29 (11) (2003) 985–995.
- [81] S. Tiwari, S. Rathore, Coupling and cohesion metrics for object-oriented software: A systematic mapping study, in: *ACM International Conference Proceeding Series*, 2018, <http://dx.doi.org/10.1145/3172871.3172878>.
- [82] S. Hosseini, B. Turhan, D. Gunarathna, A systematic literature review and meta-analysis on cross project defect prediction, *IEEE Trans. Softw. Eng.* 45 (2) (2019) 111–147, <http://dx.doi.org/10.1109/TSE.2017.2770124>.
- [83] J. Saraiva, E. Barreiros, A. Almeida, F. Lima, A. Alencar, G. Lima, S. Soares, F. Castor, Aspect-oriented software maintenance metrics: A systematic mapping study, *IET Semin. Digest* 2012 (1) (2012) 253–262, <http://dx.doi.org/10.1049/ic.2012.0033>.
- [84] E. Fregnan, T. Baum, F. Palomba, A. Bacchelli, A survey on software coupling relations and tools, *Inf. Softw. Technol.* 107 (2019) 159–178.
- [85] S. Henry, D. Kafura, K. Harris, On the relationships among three software metrics, *ACM SIGMETRICS Perform. Eval. Rev.* 10 (1) (1981) 81–88.
- [86] I. Herraiz, A.E. Hassan, Beyond lines of code: Do we need more complexity metrics, in: *Making Software: What Really Works, and Why We Believe It*, O'Reilly Media, 2010, pp. 125–141.
- [87] M.A.A. Mamun, C. Berger, J. Hansson, Correlations of software code metrics: an empirical study, in: *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement*, 2017, pp. 255–266.
- [88] E. Arisholm, D.I. Sjöberg, Evaluating the effect of a delegated versus centralized control style on the maintainability of object-oriented software, *IEEE Trans. Softw. Eng.* 30 (8) (2004) 521–534.
- [89] B.A. Kitchenham, L.M. Pickard, S.J. Linkman, An evaluation of some design metrics, *Softw. Eng. J.* 5 (1) (1990) 50–58.
- [90] R. Malhotra, K. Lata, A systematic literature review on empirical studies towards prediction of software maintainability, *Soft Comput.* 24 (21) (2020) 16655–16677, <http://dx.doi.org/10.1007/s00500-020-05005-4>.
- [91] S. Saharudin, K. Wei, K. Na, Machine learning techniques for software bug prediction: A systematic review, *J. Comput. Sci.* 16 (11) (2020) 1558–1569, <http://dx.doi.org/10.3844/JCSSP.2020.1558.1569>.
- [92] S. El-Sharkawy, N. Yamagishi-Eichler, K. Schmid, Metrics for analyzing variability and its implementation in software product lines: A systematic literature review, *Inf. Softw. Technol.* 106 (2019) 1–30, <http://dx.doi.org/10.1016/j.infsof.2018.08.015>.
- [93] S. Elmidaoui, L. Cheikhi, A. Idri, A. Abran, Empirical studies on software product maintainability prediction: A systematic mapping and review, *e-Inf. Softw. Eng. J.* 13 (1) (2019) 141–202, <http://dx.doi.org/10.5277/e-Inf190105>.
- [94] R. Malhotra, M. Khanna, Software change prediction: A systematic review and future guidelines, *e-Inf. Softw. Eng. J.* 13 (1) (2019) 227–259, <http://dx.doi.org/10.5277/e-Inf190107>.
- [95] R. Malhotra, A. Chug, Software maintainability: Systematic literature review and current trends, *Int. J. Softw. Eng. Knowl. Eng.* 26 (8) (2016) 1221–1253, <http://dx.doi.org/10.1142/S0218194016500431>.
- [96] R. Malhotra, A. Bansal, Software change prediction: A literature review, *Int. J. Comput. Appl. Technol.* 54 (4) (2016) 240–256, <http://dx.doi.org/10.1504/IJCAT.2016.080487>.
- [97] M. Riaz, T. Breaux, L. Williams, How have we evaluated software pattern application? A systematic mapping study of research design practices, *Inf. Softw. Technol.* 65 (2015) 14–38, <http://dx.doi.org/10.1016/j.infsof.2015.04.002>.
- [98] J. Murillo-Morera, C. Quesada-López, M. Jenkins, Software fault prediction: A systematic mapping study, in: *CIBSE 2015 - XVIII Ibero-American Conference on Software Engineering*, 2015, pp. 446–459.
- [99] D. Radjenović, M. Heričko, R. Torkar, A. Živković, Software fault prediction metrics: A systematic literature review, *Inf. Softw. Technol.* 55 (8) (2013) 1397–1418, <http://dx.doi.org/10.1016/j.infsof.2013.02.009>.
- [100] B. Isong, E. Obeten, A systematic review of the empirical validation of object-oriented metrics towards fault-proneness prediction, *Int. J. Softw. Eng. Knowl. Eng.* 23 (10) (2013) 1513–1540, <http://dx.doi.org/10.1142/S0218194013500484>.
- [101] B. Kitchenham, What's up with software metrics? - A preliminary mapping study, *J. Syst. Softw.* 83 (1) (2010) 37–51, <http://dx.doi.org/10.1016/j.jss.2009.06.041>.